

Module M06.C01 — Pourquoi une base et pas un classeur Excel : redondance, anomalie, concurrence, volumétrie

Analyse de données appliquée & Business Intelligence
5 partie(s) assemblée(s) · document de production

Table des matières

MODULE M06.C01 — POURQUOI UNE BASE ET PAS UN CLASSEUR EXCEL : REDONDANCE, ANOMALIE, CONCURRENCE, VOLUMÉTRIE	7
1. Objectifs du chapitre	7
2. Pourquoi cette notion est importante	7
3. Explication simple — les cinq limites du classeur	8
4. Vocabulaire essentiel	8
5. Cours approfondi — le passage du classeur au SGBD	10
5.1 Le cas fil rouge : quincaillerie_export.csv	10
5.2 Les sept défauts métier, ou pourquoi le brut n'est pas la table	11
5.3 La 3FN appliquée à quincaillerie_export.csv	11
5.4 Vue calculée et intégrité — ce qui remplace les colonnes supprimées	12
5.5 Les 3 requêtes qui ferment le module	12
5.6 Les 5 critères du choix DuckDB / SQLite / PostgreSQL	13
5.7 « Quand basculer ? » — la règle des 100 000 lignes et le verdict	13
6. Exemple concret — le verdict de la bascule	14
6.1 L'export quincaillerie_export.csv côté Excel	14
6.2 Le même fichier dans DuckDB	14
6.3 Ce qu'un SGBD ne fait pas (par construction)	14
7. Démonstration pas à pas — comment basculer, en 4 étapes	14
7.1 Importer le CSV	14
7.2 Filtrer les défauts métier	15
7.3 Définir les 7 tables en 3FN	15
7.4 Vérifier l'intégrité par les 3 requêtes	15
8. Erreurs fréquentes	15
9. Bonnes pratiques professionnelles	16
10. Exercice guidé — la lettre au stagiaire (15 min, /10)	17
11. Exercices autonomes	17
12. Correction détaillée	17
13. Mini-projet M06.P1 — « Le verdict de la bascule » (1 h)	18
14. Boîte à outils du chapitre	18
15. Résumé du chapitre	18
16. À retenir	19
17. Évaluation formative (auto-correction, 8 min)	19
MODULE M06.C02 — TABLE, COLONNE, LIGNE, SCHÉMA, TYPE, CONTRAINTE, VALEUR NULLE	20
1. Objectifs du chapitre	7
2. Pourquoi cette notion est importante	7
3. Explication simple — la table est un objet à 6 attributs	21
4. Vocabulaire essentiel	8
5. Cours approfondi — la grammaire d'une table	24
5.1 Le squelette d'un CREATE TABLE	24

5.2 Les 5 types SQL de base, et leur usage	24
5.3 NULL et la logique ternaire	24
5.4 Les 4 contraintes de colonne	24
5.5 Erreurs courantes	25
5.6 La table client du mini-projet M06.P1, en DDL DuckDB	25
6. Exemple concret — les 3 tables de référence du mini-projet M06.P1	26
6.1 Table client (vue § 5.6)	26
6.2 Table produit en DDL DuckDB	26
6.3 Table mode_paiement en DDL DuckDB	26
6.4 Insertion de 5 lignes et lecture de contrôle	26
7. Démonstration pas à pas — créer une table en 6 lignes	27
7.1 Choisir le nom	27
7.2 Lister les colonnes	27
7.3 typer chaque colonne	27
7.4 Ajouter les contraintes	27
7.5 Ordonner les contraintes de table	27
7.6 Tester en 3 requêtes	27
8. Erreurs fréquentes	15
9. Bonnes pratiques professionnelles	16
10. Exercice guidé — la table mode_paiement (15 min, /10)	28
11. Exercices autonomes	17
12. Correction détaillée	17
13. Mini-projet M06.P1 — « Les 3 tables de référence » (1 h)	29
14. Boîte à outils du chapitre	18
15. Résumé du chapitre	18
16. À retenir	19
17. Évaluation formative (auto-correction, 8 min)	19
MODULE M06.C03 — CLÉS ET RELATIONS : PRIMAIRE, ÉTRANGÈRE, CANDIDATE, COMPOSITE, 1-1, 1-N, N-M	32
1. Objectifs du chapitre	7
2. Pourquoi cette notion est importante	7
3. Explication simple — les 6 éléments d'une relation	33
4. Vocabulaire essentiel	8
5. Cours approfondi — les trois relations	35
5.1 La relation 1-N (la plus courante)	35
5.2 La relation 1-1 (la rare)	36
5.3 La relation N-M (la subtile)	36
5.4 La cardinalité en pratique — la matrice du projet M06.P	36
5.5 L'action ON DELETE — les 4 verbes	37
5.6 L'intégrité référentielle — la requête Q1 du controles_integrite.sql	37
5.7 Réparer une violation d'intégrité	37
5.8 Le MATCH SIMPLE (défaut) vs MATCH FULL	38
6. Exemple concret — les 7 FK du projet M06.P	38
6.1 DDL complet de vente (extrait du schema_3fn.sql)	38
6.2 DDL complet de ligne_vente — la N-M matérialisée	38
6.3 DDL complet de client_categorie_pref — la table de jointure	38

7. Démonstration pas à pas — lire un schéma 3FN	39
7.1 Identifier les 7 cardinalités du projet M06.P	39
7.2 Repérer les 3 CASCADE	39
7.3 Vérifier l'intégrité par Q1	39
7.4 Tester l'effet du RESTRICT sur vente.client	39
8. Erreurs fréquentes	15
9. Bonnes pratiques professionnelles	16
10. Exercice guidé — la table ligne_vente (15 min, /10)	40
11. Exercices autonomes	17
12. Correction détaillée	17
13. Mini-projet M06.P2 — « Les 7 relations du projet M06.P » (1 h)	41
14. Boîte à outils du chapitre	18
15. Résumé du chapitre	18
16. À retenir	19
17. Évaluation formative (auto-correction, 8 min)	19
MODULE M06.C04 — VIVRE AVEC DUCKDB / SQLITE / POSTGRESQL : CRÉER, INSÉRER, IMPORTER UN CSV, SE CONNECTER	44
1. Objectifs du chapitre	7
2. Pourquoi cette notion est importante	7
3. Explication simple — les 3 moteurs, en 3 phrases	45
4. Vocabulaire essentiel	8
5. Cours approfondi — vivre avec DuckDB (intégré)	47
5.1 Installer et vérifier	47
5.2 Ouvrir une base	47
5.3 Créer une table en DDL	47
5.4 Insérer des données — les 2 voies	47
5.5 Se connecter avec un client graphique	48
5.6 Sauvegarder et migrer	48
6. Cours approfondi — vivre avec SQLite (embarqué)	49
6.1 Ouvrir une base en Python	49
6.2 Importer un CSV	49
6.3 Exécuter les 3 requêtes controles_integrite.sql	49
6.4 Sauvegarder SQLite	50
7. Cours approfondi — situer PostgreSQL (cité, non exécuté)	50
7.1 Pourquoi PostgreSQL n'est pas exécuté dans cet atelier	50
7.2 Comment il fonctionne (cité)	50
7.3 Les 5 différences avec DuckDB	50
7.4 Quand choisir PostgreSQL (cité)	51
8. Exemple concret — l'import CSV en DuckDB (le geste canonique)	51
8.1 Le contexte	51
8.2 Script complet en Python	51
8.3 Le verdict attendu	52
9. Démonstration pas à pas — la chaîne DuckDB en 6 étapes	52
9.1 Installer DuckDB	52
9.2 Créer la base	52
9.3 Lire le CSV en staging	52

9.4 Insérer dans les tables cibles	52
9.5 Vérifier par les 3 requêtes	52
9.6 Sauvegarder	52
10. Erreurs fréquentes	52
11. Bonnes pratiques professionnelles	53
12. Exercice guidé — la chaîne DuckDB en 6 étapes (30 min, /15)	53
13. Exercices autonomes	54
14. Correction détaillée	54
15. Mini-projet M06.P3 — « L'import contrôlé du brut » (1 h)	54
16. Boîte à outils du chapitre	55
17. Health Check (auto-correction, 6 questions, 15 min)	55
18. Résumé du chapitre	56
19. À retenir	56
20. Évaluation formative (auto-correction, 8 min)	56
MODULE M06.C05 — LES AUTRES MONDES : NOSQL (DOCUMENT, CLÉ-VALEUR, GRAPHE, COLONNES), DATALAKE, ENTREPÔT, LAC	58
1. Objectifs du chapitre	7
2. Pourquoi cette notion est importante	7
3. Explication simple — la carte du monde des données	59
4. Vocabulaire essentiel	8
5. Cours approfondi — les 4 familles NoSQL	61
5.1 Document store (MongoDB, CouchDB)	61
5.2 Clé-valeur (Redis, DynamoDB)	62
5.3 Graphe (Neo4j, JanusGraph)	62
5.4 Colonnes larges (Cassandra, Bigtable, HBase)	62
6. Cours approfondi — datamart, datawarehouse, datalake, lakehouse	62
6.1 Le datawarehouse (l'entrepôt, ~30 ans d'histoire)	62
6.2 Le datamart (le sous-entrepôt)	63
6.3 Le datalake (le lac, ~10 ans d'histoire)	63
6.4 Le lakehouse (la combinaison ~5 ans)	63
7. Cours approfondi — les 5 mots d'entretien	63
7.1 Sharding (partitionnement horizontal)	63
7.2 Réplication (copie asynchrone)	63
7.3 Théorème CAP (Brewer)	63
7.4 ACID vs BASE (modèle transactionnel)	64
7.5 ETL vs ELT	64
8. Exemple concret — le verdict du paysage	64
9. Démonstration pas à pas — les 5 mots d'entretien	65
9.1 Préparer une réponse à « C'est quoi le sharding ? »	65
9.2 Préparer une réponse à « ACID ou BASE ? »	65
9.3 Préparer une réponse à « DWH ou datalake ? »	65
10. Erreurs fréquentes	52
11. Bonnes pratiques professionnelles	53
12. Exercice guidé — préparer 5 réponses d'entretien (30 min, /15)	66

13. Exercices autonomes	54
14. Correction détaillée	54
15. Mini-projet M06.P4 — « Le glossaire des 5 mots d’entretien » (1 h)	67
16. Boîte à outils du chapitre	55
17. Résumé du chapitre	68
18. À retenir	68
19. Évaluation formative (auto-correction, 8 min)	68

Module M06.C01 — Pourquoi une base et pas un classeur Excel : redondance, anomalie, concurrence, volumétrie

Outils comparés : classeur Excel (Microsoft 365), DuckDB 1.5.5, SQLite natif Python, PostgreSQL cité. **Durée indicative :** 5 h. **Niveau :** N2. **Prérequis :** M01.C03 (la table), M04.C01-C05 (diagnostic et qualité).

L'idée du chapitre. Le classeur Excel a accompagné toute la phase 1 (M03). Il fait semblant de suffire à tout, mais il s'effondre à la troisième table jointe et à la cinquantième millième ligne. Le module M06 installe **les raisons** pour lesquelles on passe au SGBD (système de gestion de base de données), et installe **les limites** du classeur pour qu'un chargé d'étude sache quand basculer. Le chapitre C01 fait la moitié du chemin : il pose les cinq limites, la solution SGBD, et les cinq critères du choix (calibre du C05 réinvestis). Les chapitres C02-C04 font le reste : table et schéma, clés et relations, vivre avec DuckDB et SQLite. Le chapitre C01 ferme la porte — ou plutôt l'ouvre — par les **5 critères** et le **verdict : DuckDB intégré / SQLite embarqué / PostgreSQL serveur** pour les trois cas du projet M06.P.

Base de travail — l'export quincailerie_export.csv (livre M06.P, 24 000 lignes × 42 colonnes). Le chapitre est mesuré sur ce dossier livré par `tools/dossier_M06.py` (graine 43) : qui que vous soyez, vous touchez les mêmes 24 000 lignes et les mêmes 42 colonnes. Les défauts structurels (ville_client redondante, catégorie_pref multi-valuée, taux_tva transitive, mode_paiement référence) et les défauts métier (142 montants texte, 8 doublons, 12 dates inversées, 3 clients orphelins) sont ce que le classeur Excel **ne sait pas gérer** seul — c'est l'objet du § 5.

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **énumérer** les cinq limites du classeur Excel pour un fichier multi-tables de plus de 20 000 lignes ;
- **distinguer** une base de données d'un classeur (le SGBD gère, le classeur stocke) ;
- **reconnaître** les quatre défauts structurels d'un export de caisse dénormalisé (redondance, multi-valué, dépendance transitive, référence) ;
- **ranger** DuckDB, SQLite et PostgreSQL selon les 5 critères du C05 (taille, fréquence, compétences, gouvernance, simplicité) ;
- **décider** quand basculer (la règle des 100 000 lignes de M03) et **dire non** à Excel au-delà ;
- **articuler** le passage Excel → SGBD sans drame (les 4 étapes du § 6.4).

2. Pourquoi cette notion est importante

- Le **classeur** a accompagné M01-M03 (lecture, statistiques, Excel). Il fait presque tout, et le professionnel qui s'y cantonne croit qu'il fait tout. Le module M06 installe la lucidité : le classeur fait semblant, et le SGBD fait. C'est la frontière entre l'analyste qui calcule juste et celui qui livre faux.
- L'**export de caisse** (le dossier du projet) contient 42 colonnes pour 24 000 lignes : un classeur l'ouvre, mais le **dédoublonnage clé complète** (équivalent du C04) prend 30 secondes

par recalcul. C’est la frontière des 100 000 lignes de C05 §5.1 — un fichier de 240 000 lignes ne s’ouvre pas dans Excel du tout.

- Le **passage** au SGBD n’est pas une réforme : c’est un changement de pratique. Le chargé d’étude qui passe une table de 24 000 lignes dans DuckDB continue de faire la même chose qu’avant, plus vite, plus juste, plus reproductible. C’est cette progressivité qui fait que M06 suit M05 dans le cursus, et non le remplace.
- Les **5 critères** du C05 sont réinvestis tels quels : taille, fréquence, compétences, gouvernance, et un cinquième que le module M06 introduit — la **simplicité de mise en place** (DuckDB ne demande aucune installation ; PostgreSQL demande un serveur). C’est l’objet du § 6.
- Le **verdict** du projet M06.P : DuckDB en première intention (intégré, mono-processus, gratuit), SQLite en embarqué (fichier, sans client), PostgreSQL cité mais non exécuté dans l’atelier (la même règle que Power Query pour M05). C’est le compromis du module.

3. Explication simple — les cinq limites du classeur

Le classeur Excel est **excellent** pour une table, un poste, un utilisateur. Il a **cinq limites** quand le fichier grandit et que l’équipe s’élargit. Pour chaque limite, une manifestation concrète dans le brut livré par la quincaillerie :

Limite	Manifestation dans le brut livré par la quincaillerie
Redondance	la colonne <code>ville_client</code> (col 12) recopie la ville dans chacune des 24 000 lignes. Le client 844 habite « Koudougou », et cette chaîne de 10 caractères est répétée 24 000 / 1 891 ≈ 13 fois par client. Si le client déménage, il faut changer la ville dans 13 lignes — et il en manquera toujours une (règle : une donnée, une seule place).
Anomalie de mise à jour	<code>taux_tva_courant</code> (col 21) vaut 0.18 en 2025 et 0.19 en 2026. Si la loi change le 1 ^{er} janvier, chaque vente de 2026 doit passer de 0.18 à 0.19 manuellement — sinon le calcul de TVA diverge par ligne. C’est l’objet de la 3FN (la donnée qui dépend d’une autre colonne est externalisée).
Multi-valuée	<code>categorie_pref</code> (col 23) ne porte qu’une seule catégorie par client, alors qu’un client peut en aimer 1 à 3 (bricolage + jardinage + décoration, par exemple). Le classeur force à choisir : c’est la violation de la 1FN. La parade est une table de jointure (client × catégorie).
Concurrence	deux caissières ouvrent le même classeur en même temps : Excel crée deux versions, l’« originale » et la « concurrente ». Aucune ne sait qui a la dernière mise à jour. Le SGBD verrouille la ligne, pas le fichier.
Volumétrie et performance	24 000 lignes, c’est l’instant où Excel commence à ralentir (filtre 4 secondes, recalcul 8 secondes). À 240 000 lignes, le classeur met 30 secondes à ouvrir et plante à 700 000. Le SGBD tient des milliards de lignes.

Le tableau **résume** les cinq limites, mais l’enseignement du module est plus subtil : ce n’est pas la limite la plus visible qui fait basculer, c’est le **risque** qu’elle porte. La redondance est visible (les villes sont en double), l’anomalie de mise à jour est **invisible** jusqu’au jour où le client déménage. La volumétrie est mesurable, la concurrence est ressentie au premier doublon.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
classeur workbook	— un fichier <code>.xlsx</code> qui contient une ou plusieurs feuilles ; chaque feuille est à plat, sans lien formel avec les autres.	confondre avec « base de données » : un classeur avec 5 feuilles n’est pas une base, c’est 5 tables aplaties dans 5 fichiers logiques.
base de données — database	un ensemble de tables reliées par des contraintes (clés étrangères, types, valeurs), persisté sur disque et requêtable par SQL.	croire qu’un CSV en est une : un CSV est une vue aplatie d’une base, pas une base.

Français — English	Définition simple	Piège à éviter
SGBD — Database Management System, DBMS	le logiciel qui gère la base : DuckDB, SQLite, PostgreSQL, MySQL, SQL Server, Oracle. Le SGBD stocke, indexe, verrouille, sauvegarde.	confondre avec « SQL » : le SQL est le langage (la grammaire des requêtes), le SGBD est le moteur (celui qui exécute).
DDL — Data Definition Language	le sous-ensemble du SQL qui crée la structure : CREATE TABLE, ALTER TABLE, DROP TABLE.	croire qu'il faut un outil séparé : DDL est du SQL, exécuté par le même client.
DML — Data Manipulation Language	le sous-ensemble du SQL qui touche aux données : SELECT, INSERT, UPDATE, DELETE.	confondre avec « programmation » : les UPDATE/DELETE sont transactionnels (cf. M07, §5.3 ACID).
table — table, relation	un ensemble de lignes (tuples) de même schéma, nommées par des colonnes typées. Le mot « relation » vient de la théorie (Codd, 1970) ; en pratique on dit « table ».	confondre avec « onglet » : un onglet Excel a un en-tête et des lignes, mais sans schéma ni contrainte formelle.
schéma — schema	la structure d'une base (les noms de tables, les colonnes, les types, les contraintes), distincte des données .	confondre avec « table » : une base a un schéma (ou plusieurs, séparés par namespace) et N tables.
redondance — redundancy	le fait qu'une même valeur est recopiée dans plusieurs lignes ou colonnes, au lieu d'être centralisée. C'est l'opposé de la 3FN.	confondre avec « rapidité » : la redondance accélère parfois la lecture, mais ouvre la porte à l'incohérence.
anomalie de mise à jour — update anomaly	quand une donnée redondante est mise à jour à un endroit mais pas aux autres : la base devient incohérente. C'est la conséquence de la redondance.	croire qu'on peut « mettre à jour régulièrement » : la parade est d' éliminer la redondance , pas de la compenser.
clé primaire — primary key	la colonne (ou combinaison) qui identifie une ligne de manière unique et non nulle. Elle est implicite et non modifiable dans la base.	confondre avec « identifiant métier » (ex. <code>id_ticket</code>) : la clé primaire peut être propre à la base (un <code>id_vente</code> auto-incrémenté).
clé étrangère — foreign key	une colonne qui pointe sur la clé primaire d'une autre table. La contrainte FK garantit que la valeur pointée existe.	croire qu'elle est juste « pour information » : sans la contrainte FK, la colonne n'est qu'un nombre sans lien.
intégrité référentielle — referential integrity	la garantie qu'une clé étrangère pointe sur une ligne qui existe. C'est un contrat que la base tient (ou pas).	croire qu'elle va de soi : il faut l' écrire dans le DDL (FOREIGN KEY ... REFERENCES ...).
1FN — 1st Normal Form	chaque colonne porte une valeur atomique , et pas de répétitions. Le multi-valué (cf. <code>categorie_pref</code>) la viole.	croire qu'elle est décorative : la 1FN est la condition minimale pour qu'une table soit une table relationnelle.
3FN — 3rd Normal Form	toutes les colonnes non-clés dépendent de la clé, de toute la clé, rien que de la clé. Pas de dépendance transitive. C'est l' objectif du module M06.	confondre avec « pas de redondance » : la 3FN autorise la redondance calculable (les vues).
transaction — transaction	une unité atomique de modifications (BEGIN, COMMIT, ROLLBACK), avec les propriétés ACID (atomique, cohérent, isolé, durable).	confondre avec « script » : un script peut faire 7 étapes, mais ce n'est pas une transaction (cf. M07 §5.3).
DuckDB — DuckDB	un SGBD intégré (in-process), orienté analyse, gratuit, mono-utilisateur. Il lit les CSV, Parquet, JSON, et fait du SQL ANSI.	confondre avec « PostgreSQL léger » : DuckDB n'est pas un serveur, c'est une bibliothèque qu'on appelle depuis Python ou le CLI.
SQLite — SQLite	un SGBD embarqué (fichier <code>.sqlite</code>), mono-utilisateur, ultra-léger. Utilisé sur Android, iOS, Firefox, et des millions d'applications desktop.	confondre avec « MySQL sans effort » : SQLite n'a pas de client réseau ; c'est un fichier qu'on ouvre.
PostgreSQL — PostgreSQL	un SGBD serveur, multi-utilisateurs, robuste, open source. La référence « sérieuse » pour une base de production.	croire qu'il faut l'installer pour le module : l'atelier ne peut pas exécuter PostgreSQL ; on le cite, on ne l'installe pas.

Définition. La **base de données relationnelle** est un ensemble de **tables reliées par des contraintes** (clés étrangères, types, valeurs, contrôles), persisté sur disque et requêtable par SQL via un SGBD. La différence avec un classeur Excel n'est pas la quantité de données (un classeur peut en stocker autant) mais la **forme des contraintes** : dans la base, les liens entre tables sont portés par la base elle-même (la contrainte `FOREIGN KEY`), là où dans un classeur ils sont implicites et fragiles (la jointure manuelle par copier-coller). — English : relational database.

Définition. Le **SGBD** (système de gestion de base de données) est le **moteur logiciel** qui gère la base : stockage, indexation, verrouillage, sauvegarde, exécution des requêtes SQL. Les trois SGBD du module sont DuckDB (intégré, mono-processus), SQLite (embarqué, fichier), PostgreSQL (serveur, multi-utilisateurs). Le SGBD est au service de la base, il n'est pas la base. — English : Database Management System, DBMS.

Définition. La **3FN** (3^e forme normale) est la condition qui dit : toutes les colonnes non-clés dépendent de la clé, de toute la clé, rien que de la clé. C'est l'objectif du module : pas de dépendance **transitive** (`taux_tva_courant` qui dépend de l'année, pas de la vente), pas de dépendance **partielle** (pas applicable quand la clé est simple). La 1FN ajoute « pas de multi-valué ». La 3FN est suffisante pour la plupart des cas d'usage. — English : Third Normal Form, 3NF.

Définition. La **clé étrangère** (foreign key, FK) est une **colonne** dont les valeurs sont contraintes par une **référence** vers la clé primaire d'une autre table. La contrainte FK est ce qui fait le lien : sans elle, la colonne n'est qu'un nombre qui peut pointer dans le vide (le défaut des « clients orphelins » du § 5.2). Avec elle, la base **garantit** l'intégrité référentielle. — English : foreign key, FK.

Attention. Le **classeur Excel** ne vérifie pas — `SOMME(montant_ttc)` ignore les 142 montants texte **silencieusement**, sans erreur, comme le `SUM` DuckDB autoloading (cf. C05 §5.6). Le défaut est invisible dans les deux cas, et c'est **l'import contrôlé par script** qui le détecte. Différence majeure : DuckDB documente la conversion dans les logs, Excel n'a aucun log.

Attention. Le **SGBD ne corrige pas** les défauts métier du brut — il les voit. Si l'export a 142 montants texte, ces 142 lignes sont **dans la base** au lieu d'être silencieusement ignorées. La règle du module est import + 3 requêtes = base propre : l'import filtre (`TRY_CAST`, `NOT NULL`, dates), les 3 requêtes vérifient (Q1 référentielle, Q2 montants, Q3 rejouable). Sans ce duo, la base n'est pas une base, c'est un dépôt.

5. Cours approfondi — le passage du classeur au SGBD

5.1 Le cas fil rouge : `quincaillerie_export.csv`

Le dossier du projet M06.P contient **un seul fichier** : `quincaillerie_export.csv`, 24 000 lignes, 42 colonnes, encodage UTF-8. C'est un **export de caisse** : chaque ligne est une ligne de ticket, et toutes les informations (client, produit, paiement, magasin) sont à plat. Le stagiaire qui reçoit ce fichier et doit en faire une base de données tombe sur **quatre défauts structurels** qu'il doit comprendre avant de toucher au schéma.

Défaut 1 — redondance. La colonne `ville_client` (12) répète la ville du client dans chacune de ses ventes. Si l'un des clients de l'export (par exemple un client référencé dans les 1 891 du fichier livré) a 13 achats dans le mois, la chaîne « Koudougou » est recopiée **13 fois**. Le jour où ce client déménage à Ouagadougou, il faut modifier 13 lignes — et la règle d'or (une donnée, une seule place) est violée. La parade est une **table client** séparée, et la ville y reste une fois.

Défaut 2 — multi-valué. La colonne `categorie_pref` (23) porte une **seule** catégorie préférée par client, alors qu'un client peut en avoir 1 à 3. C'est la violation de la **1FN** : une colonne devrait être atomique, mais on triche en concaténant mentalement « bricolage ; jardinage ; décora-

tion ». La parade est une **table de jointure** `client_categorie_pref(id_client, categorie)`, qui porte une ligne par couple (client, catégorie).

Défaut 3 — dépendance transitive. La colonne `taux_tva_courant` (21) vaut 0.18 en 2025 et 0.19 en 2026. Elle est portée par chaque vente, mais elle dépend de l'**année fiscale**, pas de la vente. C'est la violation de la **3FN** : une colonne non-clé ne doit dépendre que de la clé (`id_vente`), pas d'une autre colonne. La parade est une **table regle_tva(annee_fiscale, taux_tva)**, et la **vue** (cf. § 5.4) qui joint `ligne_vente` à `regle_tva` pour calculer le TTC à la lecture.

Défaut 4 — référence. La colonne `mode_paiement` (18) porte une chaîne (« ESPECES », « MOBILE_MONEY », « CARTE_BANCAIRE », « CREDIT »). Ces quatre valeurs sont figées par les usages du métier, et **elles ne changent presque jamais**. Les recopier dans chaque ligne gaspille de l'espace et ouvre la porte aux fautes de frappe (« ESPECSE »). La parade est une **table de référence** `mode_paiement(id, libelle, frais_pct, delai_encaissement_jours)`, et une **clé étrangère** `vente.id_mode REFERENCES mode_paiement(id_mode)`.

5.2 Les sept défauts métier, ou pourquoi le brut n'est pas la table

L'export contient aussi sept défauts « métier » que la **normalisation** (3FN) ne corrige pas. Ce sont les mêmes natures qu'en M05, simplifiées :

Défaut	Compteur mesuré	Effet sur le total
142 montants en texte (« 12 500 FCFA »)	<code>m06p_montants_texte_export = 142</code>	la somme directe rate 142 lignes
8 doublons exacts à la fin du fichier	<code>m06p_doublons_a_retirer = 8</code>	double comptage
12 dates inversées (jour-mois)	<code>m06p_dates_inversees_a_reparer = 12</code>	tri et coupe faussés
3 <code>id_client</code> orphelins	<code>m06p_clients_orphelins_a_ecarter = 3</code>	jointure naïve gonfle
5 % de retours (montants négatifs)	non compté dans <code>ATTENDU.json</code>	filtre « montant < 0 »
Encodage UTF-8 (sans BOM)	<code>quincaillerie_export.csv</code>	Excel l'ouvre sans message
1M cellules textuelles dans Excel	$\approx 24\,000 \times 42 = 1\,008\,000$	recalcul 30 s à 240k lignes

La **parade** est l'**import contrôlé** : un script qui charge le CSV, **vérifie chaque ligne**, rejette les violations, et **rapporte** les rejets. Le `controles_integrite.sql` du projet M06.P contient les **3 requêtes** qui vérifient, en SQL, que l'import a bien filtré ces 7 défauts. C'est l'objet du chapitre C03.

5.3 La 3FN appliquée à `quincaillerie_export.csv`

Le passage de 42 colonnes à **7 tables** est l'épreuve de la **3FN**. La règle est **purement mécanique** : 1. **Extraire** toute colonne qui dépend d'une partie de la clé (1FN) ou d'une autre colonne non-clé (3FN). 2. **Créer** une table par groupe de colonnes extraites, avec une clé primaire. 3. **Lier** par des clés étrangères.

L'application sur le brut donne sept tables, dans l'ordre où il faut les créer (les tables **référées** avant les tables **faisant référence**) :

```

1. magasin(id_magasin, libelle)           -- 3 magasins
2. mode_paiement(id_mode, libelle, frais_pct,    -- 4 modes
   delai_encaissement_jours)
3. regle_tva(annee_fiscale, taux_tva)         -- 2 règles (2025 et 2026)
4. produit(id_produit, libelle, categorie,      -- 312 produits
   sous_categorie, fournisseur,
   prix_courant_ht)
5. client(id_client, nom, prenom, telephone,    -- 1 891 clients
   email, ville, code_postal, region,
   pays, fidelite_niveau,
   fidelite_points)
6. client_categorie_pref(id_client, categorie)  -- table de jointure N-M
7. vente(id_vente, id_ticket, id_client,        -- 24 000 ventes
   id_magasin, id_vendeur, id_mode,
   date_vente, annee_fiscale, canal,
   est_promo, horodatage)
8. ligne_vente(id_vente, id_produit,            -- N lignes par vente
   quantite, prix_unitaire_ht,
   montant_ht, montant_remise_ht)
9. retour(id_retour, id_vente, motif,          -- 5 % des ventes
   montant_rembourse)

```

Les 8 colonnes supprimées (`client_nom`, `client_prenom`, `client_telephone`, `client_email`, `produit_libelle`, `produit_categorie`, `produit_sous_categorie`, `fournisseur`, `frais_pct`, `delai_encaissement_jours`, `taux_tva_courant`, `ville_client`, `categorie_pref`) sont **toutes des colonnes transitaires ou redondantes** : leur valeur est portée par la table de référence, et la valeur **calculable** (TVA, ville) est lue par jointure à la demande.

5.4 Vue calculée et intégrité — ce qui remplace les colonnes supprimées

La **vue** `ligne_vente_ttc` (du `schema_3fn.sql`) recalcule à la lecture le **montant TTC** et le **montant TVA** par jointure avec `regle_tva`. C'est l'art du relationnel : la **donnée stockée** est minimale (juste ce qui ne peut pas se calculer), la **donnée calculée** est lue par vue.

```

CREATE VIEW ligne_vente_ttc AS
SELECT lv.*,
       ROUND(lv.montant_ht * (1 + rt.taux_tva)) AS montant_ttc,
       ROUND(lv.montant_ht * rt.taux_tva) AS montant_tva
FROM ligne_vente lv
JOIN vente v ON lv.id_vente = v.id_vente
JOIN regle_tva rt ON v.annee_fiscale = rt.annee_fiscale;

```

Cette vue est **équivalente** à la lecture du brut, mais elle est **fiable** : si le taux de TVA change au 1^{er} janvier, il suffit de modifier **une ligne** de la table `regle_tva`, et toutes les ventes 2026 voient le bon taux.

5.5 Les 3 requêtes qui ferment le module

Les **3 requêtes** du `controles_integrite.sql` sont la **garantie** que la base est intègre. Chaque requête renvoie **0 ligne** quand tout va bien :

Q1 — Intégrité référentielle. Aucune vente ne pointe sur un client qui n'existe pas :

```

SELECT v.id_vente, v.id_client
FROM vente v LEFT JOIN client c ON v.id_client = c.id_client
WHERE c.id_client IS NULL;

```

Le `LEFT JOIN` ramène les ventes qui n'ont pas de correspondance ; le `WHERE c.id_client IS NULL` filtre celles qui pointent dans le vide. Attendu : **0 ligne** après import.

Q2 — Cohérence des montants. Chaque `ligne_vente.montant_ht` est bien `quantite × prix_unitaire_ht` :

```
SELECT id_vente, id_produit, quantite, prix_unitaire_ht, montant_ht,
       (quantite * prix_unitaire_ht) AS attendu
FROM ligne_vente
WHERE montant_ht != quantite * prix_unitaire_ht;
```

Attendu : **0 ligne**. Si la requête renvoie des lignes, c’est que l’import a chargé des valeurs incorrectes (typiquement des montants en texte non convertis).

Q3 — Total par catégorie. Le total `SUM(montant_ht)` par catégorie correspond au brut original :

```
SELECT p.categorie, SUM(lv.montant_ht) AS total_categorie
FROM ligne_vente lv JOIN produit p ON lv.id_produit = p.id_produit
GROUP BY p.categorie ORDER BY total_categorie DESC;
```

Attendu : **5 lignes** (Alimentaire, Bricolage, Jardinage, Quincaillerie, Décoration), avec un total qui correspond au brut original.

5.6 Les 5 critères du choix DuckDB / SQLite / PostgreSQL

Le passage Excel → SGBD n’a **pas une seule réponse**. Trois moteurs coexistent dans le module, et le choix dépend des **5 critères** suivants (les 4 critères du C05 §5.4 + 1 nouveau, la simplicité) :

Critère	DuckDB (intégré)	SQLite (embarqué)	PostgreSQL (serveur)
Taille	bon jusqu’à 50 M	bon jusqu’à 10 M	illimité
Fréquence	batch (ponctuel)	batch ou application	transactionnel
Compétences	Python ou CLI	Python ou CLI	SQL + administration
Gouvernance	fichier local	fichier local	serveur centralisé
Simplicité de mise en place	aucune (<code>pip install duckdb</code>)	aucune (natif Python)	serveur à installer

La règle du module : DuckDB en première intention (à 24 000 lignes, l’usage est analyse et DuckDB est imbattable), SQLite pour une application embarquée (un fichier qu’on copie), PostgreSQL cité pour quand l’équipe grandit (la base multi-utilisateurs, avec serveur et sauvegardes automatiques). Le module M06 enseigne DuckDB et cite SQLite/PostgreSQL, parce que c’est l’usage direct de l’analyste.

5.7 « Quand basculer ? » — la règle des 100 000 lignes et le verdict

La **règle des 100 000 lignes** (cf. M05 C05 §5.1) ne suffit pas. Il faut **trois conditions** pour basculer du classeur au SGBD :

1. **Taille** > 100 000 lignes OU **volumétrie** qui grossit (multiplication par an) ;
2. **Jointures** ≥ 3 tables (au-delà, le classeur s’effondre en RECHERCHEV imbriquées) ;
3. **Concurrence** ≥ 2 utilisateurs simultanés OU **gouvernance** qui exige une trace d’audit.

Le projet M06.P coche **les trois cases** : 24 000 lignes aujourd’hui mais 240 000 attendues (×10 en 2 ans), 7 tables à joindre (vente × client × produit × paiement × TVA × magasin × ligne_vente), et la caissière de Koudougou qui ouvre en même temps que la responsable de Bobo-Dioulasso. C’est flagrant qu’il faut un SGBD. Pour un cas limite (50 000 lignes, 1 utilisateur, 2 tables), le classeur peut encore tenir — au prix de la rigueur.

6. Exemple concret — le verdict de la bascule

6.1 L'export `quincaillerie_export.csv` côté Excel

Un classeur ouvert sur l'export : - **24 000 lignes** : filtre sur `client_ville = "Koudougou"` en 4 secondes (acceptable). - **42 colonnes** : tri par `produit_categorie` puis `client_nom` en 2 secondes (acceptable). - **Calcul** : `SOMME(montant_ttc)` ramène un total sur 24 000 lignes en 8 secondes, mais **142 lignes de montants texte** sont **ignorées** (cf. C05 §5.6) : le total est faux. Le classeur ne le dit pas. - **Dédoublonnage** : `supprimer les doublons` sur la colonne `id_ticket` en 30 secondes, mais **ne traite pas les copies partielles** (8 lignes qui ne diffèrent que sur la date — invisibles au dédoublonnage Excel).

Le verdict. Le classeur ouvre, trie, filtre, mais **ne vérifie pas** et **ne reproduit pas**. La règle est classeur pour voir, SGBD pour vérifier (cf. § 5.7).

6.2 Le même fichier dans DuckDB

Le même fichier dans DuckDB (Python ou CLI) :

```
import duckdb
con = duckdb.connect()
con.execute("CREATE TABLE export AS SELECT * FROM read_csv_auto('quincaillerie_export.csv')")
print(con.execute("SELECT COUNT(*) FROM export").fetchone())      # 24 000 lignes en 80 ms
print(con.execute("SELECT SUM(montant_ttc) FROM export").fetchone()) # somme en 200 ms
```

- 24 000 lignes lues en **80 ms** (DuckDB est ~30 × plus rapide que pandas pour l'import, et Excel ne sait même pas mesurer).
- La somme `SUM(montant_ttc)` est **faux aussi** : DuckDB autoloading convertit « 1 200 FCFA » en 1200 et les ajoute au total, mais signale la conversion dans les **logs**. Le défaut n'est plus silencieux comme dans Excel : il est visible.
- **3 requêtes d'intégrité** (cf. § 5.5) sont exécutables en quelques lignes, et chacune documente son verdict par écrit.

Le verdict. DuckDB fait plus vite ce que le classeur fait plus lentement, mais surtout il **documente** ce qu'il fait (logs) et **vérifie** ce que le classeur ne vérifie pas (les 3 requêtes).

6.3 Ce qu'un SGBD ne fait pas (par construction)

Le SGBD **n'efface pas** les défauts métier du brut — il les rend visibles. Si l'export a 142 montants texte, ils sont là, et les 142 lignes sont **dans la base** au lieu d'être silencieusement ignorées. Le SGBD **ne décide pas** non plus : `INSERT` ne fait que insérer. C'est l'import qui doit convertir, et c'est l'import qui doit rejeter les lignes non conformes. La règle du module est **import + 3 requêtes** = base propre.

7. Démonstration pas à pas — comment basculer, en 4 étapes

7.1 Importer le CSV

Dans DuckDB :

```
CREATE TABLE export_brut AS
SELECT * FROM read_csv_auto('quincaillerie_export.csv', all_varchar=true);
```

Le `all_varchar=true` force toutes les colonnes en texte — c’est l’étape « chargement brut », où rien n’est converti. La table `export_brut` reflète le fichier.

7.2 Filtrer les défauts métier

```
CREATE TABLE export_filtre AS
SELECT *
FROM export_brut
WHERE id_client IS NOT NULL          -- 3 clients fantômes écartés
  AND id_client IN (SELECT id_client FROM client) -- intégrité référentielle
  AND TRY_CAST(montant_ttc AS BIGINT) IS NOT NULL -- 142 montants texte écartés
  AND date_vente ~ '^\d{4}-\d{2}-\d{2}$';      -- 12 dates mal formées écartées
```

Le filtre **applique** les règles métier avant l’import dans la table finale.

7.3 Définir les 7 tables en 3FN

```
CREATE TABLE client (
  id_client  INTEGER PRIMARY KEY,
  nom        TEXT NOT NULL,
  prenom     TEXT NOT NULL,
  telephone  TEXT NOT NULL UNIQUE,
  email      TEXT UNIQUE,
  ville      TEXT NOT NULL,
  fidelite_niveau TEXT NOT NULL DEFAULT 'COMPTOIR'
);

CREATE TABLE vente (
  id_vente    INTEGER PRIMARY KEY,
  id_client   INTEGER NOT NULL REFERENCES client(id_client),
  id_magasin  INTEGER NOT NULL,
  id_mode     INTEGER NOT NULL,
  date_vente  DATE NOT NULL,
  annee_fiscale INTEGER NOT NULL
);
```

Et ainsi de suite pour les 7 tables.

7.4 Vérifier l’intégrité par les 3 requêtes

Reprendre `controles_integrite.sql` et l’exécuter :

```
-- Q1, Q2, Q3 (cf. § 5.5).
```

À retenir. Les 4 étapes du passage Excel → SGBD sont : **importer brut**, **filtrer les défauts**, **schématiser en 3FN**, **vérifier l’intégrité**. C’est la même séquence que M05 (transformation + dédoublonnage + jointure + verdict), appliquée à un SGBD. La différence est l’automatisation : une fois l’import écrit, il est rejouable (paramétrable par date, par magasin, etc.).

8. Erreurs fréquentes

- **Croire que le SGBD résout les défauts métier du brut.** Le SGBD les voit, mais il ne les corrige pas — c’est l’import qui filtre. Une base qui contient 142 montants texte est une **mauvaise base**, même si elle est en 3FN.

- **Confondre redondance et performance.** Parfois, dénormaliser une colonne accélère la lecture (par exemple, mettre la `ville_client` directement dans la table `vente`). Mais c'est un compromis documenté : on garde la redondance quand on sait qu'on perd la mise à jour automatique. La règle du module est 3FN par défaut, redondance calculable par vue quand le besoin est mesuré.
- **Croire que la 1FN et la 3FN sont « pour les puristes ».** Un client qui a deux catégories préférées ne tient pas dans une seule colonne. Croire qu'on peut « concaténer » (« bricolage,jardinage ») est le piège classique : ça marche à l'affichage, ça casse à la jointure.
- **Importer sans vérifier.** Le `CREATE TABLE` ne vérifie rien. Seules les **3 requêtes** vérifient. Une base qui n'a pas été testée par les 3 requêtes n'est pas une base — c'est un dépôt.
- **Vouloir tout stocker.** Stocker la TVA ou le TTC dans la base empêche la mise à jour du taux. La règle est : **stocker le minimum, calculer le reste par vue**. La TVA est dans `regle_tva`, le TTC est dans la vue `ligne_vente_ttc`.
- **Utiliser PostgreSQL pour 24 000 lignes.** C'est comme prendre un semi-remorque pour livrer une pizza : ça marche, mais c'est surdimensionné. DuckDB fait le même travail sans serveur.
- **Sous-estimer la migration des données.** Importer 24 000 lignes d'un CSV dans 7 tables prend du temps — pas en minutes, mais en logique (les défauts à filtrer, les conversions à appliquer). La règle est prévoir un import contrôlé par script, pas un copier-coller Excel.

À retenir. Les 7 erreurs ci-dessus coûtent toutes le **verdict du projet M06.P**. Le projet qui oublie de filtrer les défauts métier a un total faux ; le projet qui ne vérifie pas l'intégrité a une base qui semble correcte mais ne l'est pas. La leçon est **l'import n'est pas la base — la base, c'est l'import plus les 3 contrôles**.

9. Bonnes pratiques professionnelles

- **Documenter chaque table dans le README** (le projet M06.P E5) avec son **grain** (« une ligne = un client », « une ligne = une vente »). Le grain perdu, la table est incompréhensible.
- **Tester l'intégrité par 3 requêtes** au minimum : (1) référentielle, (2) cohérence métier, (3) rejouabilité (cf. § 5.5). Sans ces 3 requêtes, la base n'est pas livrable.
- **Paramétrer l'import** par date (`WHERE date_vente BETWEEN ...`) plutôt que par fichier. Le même script doit pouvoir réimporter le mois suivant sans modification du code.
- **Versionner le schéma SQL** dans un fichier `schema_3fn.sql` commité, pas dans un notebook interactif. La base est un artefact reproductible, pas une improvisation.
- **Mesurer** l'écart entre le total du brut et le total de la base après import. Si l'écart est 0 ou faible (les défauts rejetés), la base est cohérente. Si l'écart est important, l'import a perdu des lignes.
- **Documenter les défauts rejetés** dans le journal d'import. Le projet M06.P attend un rapport de cette forme : « 142 lignes écartées pour montant texte (cf. C05 §5.6) ; 12 lignes pour date mal formée (cf. M04 E4) ; 8 lignes pour doublon (cf. M05 C03) ; 3 lignes pour client orphelin (cf. contrôle Q1) ».

Conseil professionnel. Le verdict d'une base de données n'est pas « elle marche » ; c'est « les 3 requêtes renvoient 0 ligne ». C'est cette discipline qui distingue une base d'un fichier Excel. Et c'est ce que le projet M06.P évalue.

10. Exercice guidé — la lettre au stagiaire (15 min, /10)

Objectif. Mobiliser les 5 limites du classeur sur un cas concret.

Énoncé. Vous êtes chargé d'étude dans une enseigne de distribution. Le directeur commercial vous remet un fichier `ventes_export.csv` (15 colonnes, 50 000 lignes, fait à partir du logiciel de caisse). Il dit : « tu n'as qu'à l'ouvrir dans Excel, ça doit suffire. » Vous avez 5 minutes pour répondre par mail : faut-il ouvrir dans Excel, basculer dans DuckDB, ou autre chose ?

Barème (/10).

Critère	Points
Cite au moins 3 des 5 limites du classeur	2
Identifie la limite décisive (volumétrie vs jointure vs concurrence)	2
Donne un verdict motivé (Excel ou DuckDB ou autre)	3
Chiffre les alternatives (combien de temps dans Excel, combien dans DuckDB)	2
Cite les 3 requêtes d'intégrité attendues	1
Total	10

Dans les faits. La réponse attendue : « Le fichier a 50 000 lignes (sous le seuil 100k), 1 table, pas de jointure multi-table visible, mono-utilisateur (votre service). Vous pouvez commencer dans Excel pour voir, basculer dans DuckDB dès qu'une deuxième table arrive (par exemple quand le service qualité veut croiser avec un fichier de lots). Les 3 requêtes d'intégrité attendues : Q1 référentielle, Q2 montants, Q3 total rejouable. » Le barème teste la **capacité à dire non à Excel** sans le dire par principe — c'est l'enseignement du § 5.7.

11. Exercices autonomes

- **Exercice 11.1 (15 min).** Citez les **5 limites du classeur Excel** et donnez, pour chacune, un chiffre du projet M06.P qui la rend visible.
- **Exercice 11.2 (30 min).** Sur le `quincaillerie_export.csv` téléchargé, ouvrez-le dans Excel, comptez le temps d'ouverture et le temps de filtre sur `ville_client = "Koudougou"`. Puis exécutez la même opération dans DuckDB et notez les temps. Commentez l'écart en 3 lignes.
- **Exercice 11.3 (45 min).** Identifiez, dans le brut livré, **un** défaut structurel qui n'est **pas** listé au § 5.1 (par exemple `montant_remise_ht` qui dépend d'une promotion et non de la vente). Rédigez le DDL de la table qui l'absorbe.
- **Exercice 11.4 (60 min, optionnel).** Réécrivez le verdict du § 5.5 en **PostgreSQL** plutôt qu'en DuckDB. Cherchez 3 différences syntaxiques et notez-les (par exemple : `TRY_CAST` DuckDB vs `CAST` PostgreSQL, `IS NOT DISTINCT FROM`, `ON CONFLICT`).

12. Correction détaillée

- **Exercice 11.1.** Limites : (1) redondance — la colonne `ville_client` est recopiée 13 × par client (`m06p_clients_uniques_references` = 1 891, 24 000 lignes) ; (2) anomalie de mise à jour — la TVA change tous les ans, 24 000 lignes à mettre à jour ; (3) multi-valué — `categorie_pref` ne porte qu'**une** catégorie par client ; (4) concurrence — la caissière de Koudougou et la responsable de Bobo-Dioulasso ouvrent en même temps ; (5) volumétrie — 24 000 lignes, c'est la frontière où Excel ralentit (8 secondes par recalcul).

- **Exercice 11.2.** Les temps attendus : Excel $\approx$ 4-8 secondes pour le filtre, DuckDB $\approx$ 80 ms pour l'import + 50 ms pour le filtre. **Ratio $\approx$ 100 x.** Si la machine est plus lente, le ratio reste du même ordre.
- **Exercice 11.3.** Un défaut **non listé** : `montant_remise_ht` (col 30) est nul dans 90 % des lignes, et porte 10 000 FCFA dans 10 % des lignes. Il dépend de la promotion (col 35 `est_promo`), pas de la vente. La parade est soit une table `promotion(id_promo, libelle, montant_remise)`, soit une simple **règle métier** dans le README (la remise est une constante de 10 000 pour toute vente en `est_promo = 1`). Le DDL attendu : `CREATE TABLE promotion (id_promo INTEGER PRIMARY KEY, libelle TEXT, montant_remise INTEGER NOT NULL CHECK (montant_remise > 0));` et `vente.est_promo` → pas de FK directe (les promos changent).
- **Exercice 11.4.** Trois différences DuckDB → PostgreSQL : (a) `TRY_CAST DuckDB → CAST ou (expression)::type PostgreSQL`, sans gestion d'erreur ; (b) `IS NOT DISTINCT FROM` est supporté par PostgreSQL (et DuckDB), pas par tous les SGBD ; (c) `ON CONFLICT (id) DO UPDATE` est PostgreSQL natif depuis 9.5 (upsert), DuckDB l'a aussi depuis 0.9. Le verdict : DuckDB est suffisamment compatible PostgreSQL pour le module, mais l'atelier ne fait pas l'aller-retour.

13. Mini-projet M06.P1 — « Le verdict de la bascule » (1 h)

Énoncé. Rédigez **1 page maximum** (PDF ou Markdown) qui répond, en phrases, à la question : « Le fichier `quincaillerie_export.csv` doit-il basculer dans un SGBD ? Si oui, lequel des trois (DuckDB / SQLite / PostgreSQL) ? »

Critères de réussite.

1. Les 5 limites du classeur sont citées **avec un chiffre du projet** chacune.
2. Les **3 critères limitants** (volumétrie, jointure, concurrence) sont **identifiés** sur le cas.
3. Le verdict (Excel / DuckDB / SQLite / PostgreSQL) est **motivé** par les 5 critères du § 5.6.
4. Une **règle de bascule** est formulée (« si le fichier double en 1 an, basculer dans DuckDB »).

Barème (/10). 2 points par critère sauf le 4 (règle de bascule) qui vaut 4 points.

14. Boîte à outils du chapitre

Boîte à outils. Six objets, un seul but — décider quand basculer du classeur au SGBD :

1. **Le tableau des 5 limites** (§ 5.1) — redondance, anomalie, multi-valué, concurrence, volumétrie.
2. **Le tableau des 7 défauts** (§ 5.2) — quatre structurels et trois métier, avec leur compteur dans `m06p_*`.
3. **Les 3 règles de bascule** (§ 5.7) — taille > 100 000 OU jointures ≥ 3 OU concurrence ≥ 2 .
4. **Le schéma des 7 tables** (§ 5.3) — la cible 3FN, à valider par le projet.
5. **La vue `ligne_vente_ttc`** (§ 5.4) — l'exemple type de « donnée calculée, pas stockée ».
6. **Les 3 requêtes d'intégrité** (§ 5.5) — Q1 référentielle, Q2 montants, Q3 rejouable.

15. Résumé du chapitre

- Le classeur a **5 limites** (§ 5.1) : redondance, anomalie de mise à jour, multi-valué, concurrence, volumétrie.

- La base de données est un **ensemble de tables reliées par des contraintes**, gérées par un SGBD (DuckDB, SQLite, PostgreSQL).
- L'export `quincaillerie_export.csv` (24 000 lignes × 42 colonnes) contient **4 défauts structuraux** (§ 5.1) — redondance de `ville_client`, multi-valué de `categorie_pref`, dépendance transitive de `taux_tva_courant`, référence de `mode_paiement` — et **4 défauts métier** (§ 5.2).
- La 3FN appliquée au brut donne **7 tables** (§ 5.3) : `magasin`, `mode_paiement`, `regle_tva`, `produit`, `client`, `client_categorie_pref`, `vente`, `ligne_vente` (+ `retour`).
- Les **3 requêtes** d'intégrité (§ 5.5) ferment le module : Q1 référentielle, Q2 cohérence des montants, Q3 jouable par catégorie.
- Les **5 critères du choix** SGBD (§ 5.6) : DuckDB en première intention, SQLite pour embarqué, PostgreSQL cité pour multi-utilisateurs.

16. À retenir

À retenir. Le SGBD ne résout pas les défauts du brut — il les rend visibles. C'est l'import qui filtre, et c'est l'import qui rejette. La 3FN est l'objectif, le `schema_3fn.sql` est la cible, les 3 requêtes sont la garantie. Le verdict est « les 3 requêtes renvoient 0 ligne », pas « la base fonctionne ». C'est cette discipline qui distingue un SGBD d'un classeur.

À retenir. La bascule Excel → SGBD a **3 critères limitants** : taille > 100 000 lignes, jointures ≥ 3, concurrence ≥ 2. Tant qu'aucun n'est franchi, le classeur tient ; dès qu'un seul l'est, basculer. Pour 24 000 lignes et 7 tables, **DuckDB** est le bon choix ; pour 250 millions et 200 utilisateurs simultanés, **PostgreSQL** l'est. Le critère taille du C05 §5.6 devient volumétrie + concurrence — la gouvernance locale ne suffit plus à elle seule.

17. Évaluation formative (auto-correction, 8 min)

Question 1. Citez les 5 limites du classeur Excel. (Réponse : redondance, anomalie de mise à jour, multi-valué, concurrence, volumétrie.)

Question 2. Quels sont les 4 défauts structurels du `quincaillerie_export.csv` ? (Réponse : (1) `ville_client` redondante, (2) `categorie_pref` multi-valué, (3) `taux_tva_courant` en dépendance transitive, (4) `mode_paiement` en référence à externaliser.)

Question 3. Combien de tables le passage en 3FN produit-il dans le projet M06.P ? (Réponse : `m06p_tables_3fn_cible` = 7.)

Question 4. Pourquoi la **vue** `ligne_vente_ttc` est-elle préférable à une colonne stockée ? (Réponse : la TVA change chaque année ; si elle est stockée, il faut modifier 24 000 lignes ; si elle est en vue, il suffit de modifier une ligne de `regle_tva`.)

Question 5. Citez les 3 critères qui font basculer du classeur au SGBD. *(Réponse : (a) taille

100 000 lignes, (b) ≥ 3 tables à joindre, (c) ≥ 2 utilisateurs simultanés.)*

Question 6. Quel SGBD pour 240 000 lignes multi-utilisateurs ? (Réponse : PostgreSQL — le seul des trois qui tienne le multi-utilisateurs avec un serveur.)

Question 7. Que renvoient les 3 requêtes `controles_integrite.sql` quand la base est intègre ? (Réponse : Q1 → 0 ligne, Q2 → 0 ligne, Q3 → 5 lignes (les 5 catégories, avec leur total).)

Module M06.C02 — Table, colonne, ligne, schéma, type, contrainte, valeur nulle

Outils : DuckDB 1.5.5 (intégré), SQLite natif Python, PostgreSQL cité. Durée indicative : 6 h. Niveau : N2. Prérequis : M06.C01 + M01.C03.

L'idée du chapitre. Le C01 a posé **pourquoi** une base : redondance, anomalie, concurrence, volumétrie. Le C02 pose **de quoi** une base est faite : table, colonne, ligne, schéma, type, contrainte. Le vocabulaire du C02 est **transverse** : il vaut pour DuckDB, SQLite, PostgreSQL et au-delà — un professionnel qui le maîtrise passe d'un SGBD à l'autre sans réapprendre. Le chapitre **aligne le langage** (les 5 types de base, NULL n'est ni 0 ni vide, les 4 contraintes de colonne) et installe le **mini-projet M06.P1** : 3 tables de référence créées en DDL DuckDB. Le verdict du C02 est « **la table est un objet à 6 attributs** » : nom, colonnes typées, clé primaire, contraintes, insertion atomique, lecture en bloc.

Base de travail — le projet M06.P. C02 réutilise le `quincaillerie_export.csv` (24 000 lignes × 42 colonnes, livré par `tools/dossier_M06.py`), mais ne crée **aucune table projet** : le projet est l'affaire du C03 (clés + relations) et du mini-projet C03 §10. C02 ne crée que les **3 tables de référence** du mini-projet M06.P1 : `client`, `produit`, `mode_paiement`. Ces 3 tables sont le squelette du schéma 3FN que le C03 et le projet M06.P assemblent.

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **définir** les 6 attributs d'une table : nom, schéma, colonnes, types, contraintes, lignes ;
- **distinguer** les 5 types SQL de base (numérique, texte, date, booléen, binaire) et leurs variantes DuckDB ;
- **comprendre** que NULL n'est ni 0 ni vide, et lire la logique ternaire (TRUE, FALSE, UNKNOWN) ;
- **utiliser** les 4 contraintes de colonne (NOT NULL, UNIQUE, CHECK, DEFAULT) et **tester** leur effet ;
- **créer** 3 tables de référence en DDL DuckDB, **insérer** 5 lignes, **vérifier** par requête de lecture ;
- **reconnaître** les erreurs courantes (NOT NULL violé, UNIQUE dupliqué, CHECK rejeté, NULL non géré).

2. Pourquoi cette notion est importante

- La **table** est l'unité de base du relationnel. Tout le reste (schéma, contraintes, requêtes) en découle. C'est l'objet que le professionnel doit savoir créer, lire, modifier, détruire. Le C02 installe la **création** ; le C03 installe les **relations** ; le C04 installe la **vie en production**.
- Les **5 types** (numérique, texte, date, booléen, binaire) couvrent 95 % des cas. DuckDB en ajoute 9 (HUGEINT, UUID, JSON, STRUCT, LIST, etc.) qui sont rarement nécessaires. Le C02 enseigne les 5, pas les 14 : c'est le **profil minimal** d'un analyste.
- NULL est le **piège statistique** n°1 : SUM(col) ignore les NULL, COUNT(col) aussi, mais COUNT(*) les compte. Sans comprendre NULL, on **produit des chiffres faux** sans s'en rendre compte. Le M01.B02 l'a enseigné pour le tableur ; le C02 l'enseigne pour le SQL.

- Les **4 contraintes de colonne** sont ce qui distingue la table d'un fichier texte. Sans contraintes, on peut écrire 1 client avec un `telephone` à 'abc' ou un `id_client` en double. Avec, le SGBD refuse. C'est la **discipline** du relationnel.
- Le **mini-projet M06.P1** (3 tables de référence en DDL DuckDB) est l'épreuve : il passe si les tables se créent, si les contraintes rejettent les violations, si les 5 lignes insérées se lisent. C'est avant le projet M06.P, qui assemble ces 3 tables aux 4 autres (vente, ligne_vente, retour, client_categorie_pref).

3. Explication simple — la table est un objet à 6 attributs

Une table est un objet à 6 attributs, **dans cet ordre** :

1. **Nom** — `client`, `produit`, `vente`, etc. Le nom est immuable : on ne renomme pas sans casser les requêtes.
2. **Schéma** — la liste ordonnée des colonnes, chacune avec son **nom**, son **type**, et ses **contraintes**. Le schéma est distinct des données : la table `client` a 11 colonnes, qu'elle contienne 0 ligne ou 1 891.
3. **Colonnes typées** — chaque colonne a un **type** (`INTEGER`, `TEXT`, `DATE`, etc.) qui contraint la valeur (DuckDB rejette 'abc' dans un `INTEGER`).
4. **Contraintes** — les **règles** que les données doivent suivre (`PRIMARY KEY`, `NOT NULL`, `UNIQUE`, `CHECK`, `FOREIGN KEY`).
5. **Lignes** — les **données elles-mêmes**, qui respectent le schéma et les contraintes. Une ligne qui viole une contrainte est **refusée** à l'insertion.
6. **Clé primaire** — la colonne (ou combinaison) qui identifie la ligne. Sa présence est **facultative** d'un point de vue syntaxe, mais **obligatoire** d'un point de vue métier.

Le **résumé en une phrase** : « Une table, c'est un nom, un schéma typé, des contraintes, des lignes qui les respectent, et une clé pour les compter. »

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
table — table, relation	un ensemble de lignes (tuples) de même schéma, nommées par des colonnes typées.	confondre avec « onglet » : un onglet Excel a un entête et des lignes, mais sans schéma ni contrainte formelle.
colonne — column, attribute	— un slot typé dans la table. Chaque ligne porte une valeur pour chaque colonne.	croire que l'ordre des colonnes compte : il ne compte pas pour la logique (on les nomme), mais l'ordre physique peut affecter la performance.
ligne — row, tuple	une valeur pour chaque colonne, qui respecte le schéma et les contraintes.	confondre avec « enregistrement » : une ligne est typée par ses colonnes, un enregistrement est juste un paquet de données.
schéma — schema	— la structure d'une base (les noms de tables, les colonnes, les types, les contraintes), distincte des données .	confondre avec « table » : une base a un schéma (ou plusieurs namespaces) et N tables.
type — data type	la famille des valeurs qu'une colonne peut prendre (<code>INTEGER</code> , <code>TEXT</code> , <code>DATE</code> , <code>BOOLEAN</code> , etc.).	confondre avec « format » : le type est logique (DuckDB sait que c'est une date), le format est visuel (JJ/MM/AAAA est l'affichage).
NULL — null value	une absence de valeur , distincte de 0, de '' et de 'NA'. C'est le marqueur « pas de réponse ».	confondre avec 0 ou vide : <code>NULL + 1</code> n'est pas 1, c'est <code>NULL</code> ; la logique ternaire (<code>TRUE/FALSE/UNKNOWN</code>) traite <code>NULL</code> comme inconnu.
NOT NULL	contrainte qui interdit <code>NULL</code> sur la colonne. C'est l'engagement que la colonne porte toujours une valeur.	croire que c'est facultatif : <code>NOT NULL</code> rend la colonne porteuse de sens (une date manquante n'est pas <code>NULL</code> silencieuse).

Français — English	Définition simple	Piège à éviter
UNIQUE	contrainte qui interdit les doublons sur la colonne (ou la combinaison). C'est l'engagement que la colonne identifie de manière unique.	confondre avec PRIMARY KEY : UNIQUE accepte NULL, PRIMARY KEY non.
CHECK	contrainte qui valide une condition sur la valeur (par exemple <code>montant > 0</code> , ou <code>taux IN (0.18, 0.19)</code>).	croire qu'elle est universelle : CHECK peut être contournée par certaines opérations (COPY, LOAD DATA).
DEFAULT	la valeur par défaut quand l'insertion ne précise pas la colonne. C'est l'engagement sur la valeur en cas d'oubli.	confondre avec NULL implicite : DEFAULT n'est pas NULL, c'est une valeur que vous choisissez.

Attention. **UNIQUE n'est pas PRIMARY KEY** — c'est un piège classique, surtout en SQLite où **UNIQUE** accepte plusieurs **NULL**. Si vous voulez une ligne par valeur, sans **NULL**, utilisez **PRIMARY KEY** ou **UNIQUE NOT NULL**. Et si la colonne peut être **NULL**, dites-le (**UNIQUE** accepte alors des **NULL** non-dupliqués, ce qui veut dire « plusieurs lignes peuvent avoir la même absence » — c'est rarement ce qu'on attend).

Attention. **CHECK n'est pas une règle métier** — c'est un **garde-fou syntaxique**. La règle du module est **CHECK** valide l'évidence, pas la complexité. Un **CHECK** (montant > 0) rate les retours ; un **CHECK** (montant != 0) est correct mais **inutile** (tout 0 est probablement une erreur de saisie, pas une saisie vraie) ; un **CHECK** (montant > -1 000 000 AND montant < 1 000 000) est une plage par défaut, pas une règle métier. Pour les invariants métier, le code applicatif + les 3 requêtes `controles_integrite.sql` du C01 sont plus adaptés.

Dans les faits. En production, **la moitié des bugs de base viennent des contraintes oubliées**. Une table `email` sans **UNIQUE** accepte 50 clients avec le même e-mail, et c'est visible quand on essaie d'envoyer la newsletter (« 50 exemplaires à la même adresse »). Une table `prix_unitaire` sans **CHECK** (prix > 0) accepte un prix à -1000, et c'est visible quand le calcul de marge donne un résultat aberrant. Le test qui ferme le chapitre est **l'insertion négative** : insérer une ligne qui devrait être refusée, et vérifier qu'elle l'est. C'est l'épreuve du mini-projet M06.P1 §13. | **INTEGER** | le type **entier signé** sur 32 bits (de -2 147 483 648 à 2 147 483 647). Pour aller au-delà : **BIGINT**. | croire qu'il gère les décimales : pour les décimales, c'est **DOUBLE**, **DECIMAL** ou **NUMERIC**. | | **BIGINT** | le type **entier signé** sur 64 bits (de -9 223 372 036 854 775 808 à 9 223 372 036 854 775 807). Le défaut dans beaucoup de SGBD. | confondre avec **HUGEINT** (128 bits, DuckDB) : pour 99 % des cas, **BIGINT** suffit. | | **TEXT** | le type **chaîne** de longueur variable, sans limite stricte (DuckDB ne fixe pas de limite ; PostgreSQL impose une limite pratique). | croire que **VARCHAR(255)** est obligatoire : c'est l'habitude de l'époque mainframe, pas une nécessité. | | **DATE** | le type **date calendaire** (AAAA-MM-JJ), distinct de **TIMESTAMP** (qui ajoute l'heure). | confondre avec le format : `DATE '2025-03-15'` est un littéral date, `'15/03/2025'` est une chaîne. | | **BOOLEAN** | le type **booléen** à 3 valeurs (**TRUE**, **FALSE**, **NULL** en SQL ANSI ; DuckDB utilise **TRUE/FALSE/NULL**). | croire que c'est 0/1 : en SQL, c'est **TRUE/FALSE/NULL**, pas 0/1/2. | | **PRIMARY KEY** | la **clé primaire** : la colonne (ou combinaison) qui identifie la ligne. Implique **NOT NULL** + **UNIQUE**. | croire qu'elle est obligatoire : techniquement non, mais sans elle la base ne sait pas compter les lignes. |

Définition. Une **contrainte** est une **règle** que les données doivent suivre, **portée par la base elle-même** (pas par le code applicatif). Les 4 contraintes de colonne du C02 sont **NOT NULL**, **UNIQUE**, **CHECK**, **DEFAULT**. Les contraintes de **table** (cf. C03) sont **PRIMARY KEY** et **FOREIGN KEY**. Une contrainte **refuse** l'insertion qui la viole — c'est ce qui distingue une base d'un fichier texte. — English : constraint.

Définition. Le **type SQL** est la **famille** des valeurs qu'une colonne accepte. Les 5 types de base sont : numérique (**INTEGER**, **BIGINT**, **DOUBLE**, **DECIMAL**), texte (**TEXT**, **VARCHAR**), date (**DATE**, **TIMESTAMP**), booléen (**BOOLEAN**), binaire (**BLOB**). DuckDB en ajoute 9 (**JSON**, **UUID**, **LIST**, **STRUCT**, etc.) qui sont rarement nécessaires. Le type est logique : il valide à l'insertion (le SGBD refuse `'abc'` dans un **INTEGER**) et opère sur les valeurs (+ sur des **INTEGER** fait de l'arithmétique, pas de la concaténation). — English : data type.

Définition. La valeur **NULL** représente l'**absence de valeur**. Elle est distincte de 0, de '' (chaîne vide) et de 'NA'. La logique ternaire SQL (**TRUE**, **FALSE**, **UNKNOWN**) traite **NULL** comme inconnu : **NULL = NULL** n'est pas **TRUE**, c'est **UNKNOWN**. Le test juste est **IS NULL** / **IS NOT NULL**. Les fonctions d'agrégat **ignorent** **NULL** par défaut : `SUM(col)` produit **NULL** si toutes les valeurs sont **NULL**, et ignore les **NULL** sinon (sans compter la ligne). — English : null value.

Définition. Le **DDL** (Data Definition Language) est le sous-ensemble du SQL qui crée, modifie, et supprime la **structure** : **CREATE TABLE**, **ALTER TABLE**, **DROP TABLE**, **CREATE INDEX**, **CREATE VIEW**. Le **DML** (Data Manipulation Language) touche aux **données** : **SELECT**, **INSERT**, **UPDATE**, **DELETE**. Le **DCL** (Data Control Language) gère les **droits** : **GRANT**, **REVOKE**. Le C02 enseigne le DDL ; le C03 et le C07 enseignent le DML. — English : Data Definition Language.

5. Cours approfondi — la grammaire d'une table

5.1 Le squelette d'un CREATE TABLE

La syntaxe canonique en DuckDB / PostgreSQL / SQLite est :

```
CREATE TABLE nom_table (
    nom_col1 TYPE CONTRAINTES,
    nom_col2 TYPE CONTRAINTES,
    ...
    PRIMARY KEY (col1, col2),          -- contrainte de table
    FOREIGN KEY (col3) REFERENCES autre_table(col3) -- contrainte de table (cf. C03)
);
```

Le **nom** est suivi des **colonnes**, chacune avec son **type** et ses **contraintes** (de colonne). Les contraintes de **table** (PRIMARY KEY, FOREIGN KEY) sont à la fin.

5.2 Les 5 types SQL de base, et leur usage

Type	Usage courant	Piège
INTEGER BIGINT	/ compteurs, identifiants, montants entiers (en FCFA)	croire que ça gère les décimales
DOUBLE DECIMAL	/ montants avec décimales, mesures physiques	DOUBLE perd en précision ; DECIMAL(p, s) est exact
TEXT	noms, libellés, descriptions	confondre avec VARCHAR(n) (longueur fixe)
DATE TIMESTAMP	/ dates de vente, dates de naissance	DATE '2025-03-15' est un littéral ; '15/03/2025' est une chaîne
BOOLEAN	flags (est_promo, est_retour)	en SQL c'est TRUE/FALSE/NULL, pas 0/1/NULL

DuckDB **étend** ces 5 types avec HUGEINT (128 bits), UUID, JSON, STRUCT, LIST, MAP, ENUM. Pour 95 % des cas d'usage, les 5 de base suffisent. Le projet M06.P n'utilise que INTEGER, TEXT, DATE, BOOLEAN, BIGINT (pour les montants en entier FCFA).

5.3 NULL et la logique ternaire

En SQL, NULL est une **valeur spéciale** qui signifie « **je ne sais pas** ». Ce n'est pas 0, ce n'est pas vide, ce n'est pas 'NA'. Les **comparaisons** avec NULL retournent UNKNOWN, pas TRUE ou FALSE :

```
SELECT NULL = 0;          -- NULL (UNKNOWN, pas FALSE)
SELECT NULL = NULL;       -- NULL (UNKNOWN, pas TRUE)
SELECT NULL IS NULL;      -- TRUE (le test juste)
SELECT NULL IS NOT NULL;  -- FALSE (le test juste)
```

La **logique ternaire** (TRUE, FALSE, UNKNOWN) affecte les agrégats. SUM(col) ignore les NULL, COUNT(col) aussi, mais COUNT(*) les compte. La règle du module est toujours tester IS NULL / IS NOT NULL, jamais = NULL (qui retourne UNKNOWN).

5.4 Les 4 contraintes de colonne

Contrainte	Effet	Exemple
NOT NULL	la colonne ne peut pas être NULL	telephone TEXT NOT NULL
UNIQUE	la colonne (ou combinaison) n'a pas de doublon	id_client INTEGER UNIQUE

Contrainte	Effet	Exemple
CHECK	la valeur doit satisfaire une condition	<code>prix_courant_ht INTEGER CHECK (prix_courant_ht > 0)</code>
DEFAULT	la valeur par défaut si l'insertion omet la colonne	<code>fidelite_niveau TEXT DEFAULT 'COMPTOIR'</code>

Une contrainte **PRIMARY KEY** est la combinaison **NOT NULL + UNIQUE** appliquée à la colonne désignée. Le C03 l'approfondit.

5.5 Erreurs courantes

Erreur	Symptôme	Parade
Oublier <code>NOT NULL</code>	la colonne est <code>NULL</code> par défaut	ajouter <code>NOT NULL</code> à la création (sinon, recréer la table)
Mettre <code>VARCHAR(255)</code> sans raison	les chaînes de plus de 255 caractères sont tronquées silencieusement	utiliser <code>TEXT</code> (DuckDB) ou <code>VARCHAR(N)</code> avec un N réaliste
<code>CHECK (montant_ttc > 0)</code> sur des retours	les retours (montants négatifs) sont refusés	<code>CHECK (montant_ttc != 0)</code> ou pas de <code>CHECK</code> (la règle métier est dans le code applicatif)
Comparer avec <code>= NULL</code>	retourne <code>UNKNOWN</code> , jamais <code>TRUE</code>	utiliser <code>IS NULL / IS NOT NULL</code>
Insérer des dates en <code>JJ/MM/AAAA</code>	DuckDB autoload devine (parfois faux)	utiliser <code>DATE 'AAAA-MM-JJ'</code> ou <code>STRPTIME(col, '%d/%m/%Y')</code>
Oublier <code>UNIQUE</code> sur email	plusieurs clients partagent le même e-mail	ajouter <code>UNIQUE</code> (et migrer les doublons d'abord)

5.6 La table `client` du mini-projet M06.P1, en DDL DuckDB

```
CREATE TABLE client (
  id_client INTEGER PRIMARY KEY,
  nom TEXT NOT NULL,
  prenom TEXT NOT NULL,
  telephone TEXT NOT NULL UNIQUE,
  email TEXT UNIQUE,
  n'en a pas -- nullable : un client comptoir
  ville TEXT NOT NULL,
  code_postal TEXT,
  region TEXT,
  pays TEXT NOT NULL DEFAULT 'Burkina Faso',
  fidelite_niveau TEXT NOT NULL DEFAULT 'COMPTOIR'
  CHECK (fidelite_niveau IN ('COMPTOIR','BRONZE','OR','PLATINE')),
  fidelite_points INTEGER NOT NULL DEFAULT 0 CHECK (fidelite_points >= 0)
);
```

Lecture de la table : - **id_client** : la clé primaire, `INTEGER`, identifie chaque ligne. - **nom** et **prenom** : `TEXT NOT NULL`, on refuse un client sans nom. - **telephone** : `TEXT NOT NULL UNIQUE`, on refuse un téléphone vide ET un doublon. - **email** : `TEXT UNIQUE` (sans `NOT NULL`), un client comptoir peut ne pas avoir d'e-mail ; mais deux clients ne partagent pas le même e-mail. - **ville** : `TEXT NOT NULL`, on refuse un client sans ville. - **pays** : `TEXT NOT NULL DEFAULT 'Burkina Faso'`, valeur par défaut pour la quincaillerie locale. - **fidelite_niveau** : `CHECK` qui contraint à 4 valeurs admises. - **fidelite_points** : `CHECK (>= 0)`, on refuse les points négatifs.

6. Exemple concret — les 3 tables de référence du mini-projet M06.P1

6.1 Table `client` (vue § 5.6)

C'est la table **principale** du projet M06.P — 1 891 lignes à terme, et la **cible** des clés étrangères de vente, retour, client_categorie_pref. La clé primaire est `id_client`, et l'**unicité** est sur `telephone` (clé métier, pas technique).

6.2 Table `produit` en DDL DuckDB

```
CREATE TABLE produit (
  id_produit      INTEGER PRIMARY KEY,
  libelle         TEXT      NOT NULL,
  categorie       TEXT      NOT NULL,
  sous_categorie TEXT      NOT NULL,
  fournisseur     TEXT      NOT NULL,
  prix_courant_ht INTEGER NOT NULL CHECK (prix_courant_ht > 0),
  UNIQUE (libelle, fournisseur)      -- un produit est unique par (nom, fournisseur)
);
```

Cette table respecte la **3FN** : pas de transitivité, pas de multi-valué. Le `prix_courant_ht` est l'attribut propre du produit (pas d'une autre entité).

6.3 Table `mode_paiement` en DDL DuckDB

```
CREATE TABLE mode_paiement (
  id_mode          INTEGER PRIMARY KEY,
  libelle          TEXT      NOT NULL UNIQUE,
  frais_pct        REAL      NOT NULL DEFAULT 0 CHECK (frais_pct >= 0),
  delai_encaissement_jours INTEGER NOT NULL DEFAULT 0
);
```

Les 4 valeurs canoniques (`ESPECES`, `MOBILE_MONEY`, `CARTE_BANCAIRE`, `CREDIT`) sont insérées une fois, et la table `vente` y fait référence par `id_mode`. C'est l'**externalisation de la référence** du § 1.4.

6.4 Insertion de 5 lignes et lecture de contrôle

```
INSERT INTO client (id_client, nom, prenom, telephone, email, ville, fidelite_niveau) VALUES
(0, 'COMPTOIR', 'CLIENT', '0000000000', NULL,
'Koudougou', 'COMPTOIR'),
(1, 'Nom0001', 'Prenom0001', '+226 78 12 34 56 78', 'p1.n1@example.bf',
'Koudougou', 'OR'),
(2, 'Nom0002', 'Prenom0002', '+226 78 12 34 56 79', 'p2.n2@example.bf',
'Bobo-Dioulasso', 'BRONZE'),
(3, 'Nom0003', 'Prenom0003', '+226 78 12 34 56 80', NULL,
'Ouagadougou', 'COMPTOIR'),
(4, 'Nom0004', 'Prenom0004', '+226 78 12 34 56 81', 'p4.n4@example.bf',
'Banfora', 'PLATINE');

SELECT COUNT(*) AS nb_clients FROM client;           -- 5
SELECT * FROM client WHERE fidelite_points > 0;      -- 1 si on a inséré des
points
SELECT * FROM client WHERE fidelite_niveau = 'COMPTOIR'; -- 2
SELECT * FROM client WHERE email IS NULL;           -- 2
```

Ce mini-projet **valide** la création : si DuckDB renvoie 5 pour `COUNT(*)`, si les contraintes refusent les violations (test : `INSERT INTO client (id_client, nom, prenom, telephone) VALUES (5, NULL, 'X', '+226 X')` — `NOT NULL` violé sur `nom`), si les tris par `fidelite_niveau` renvoient 2 lignes, alors la table est **opérationnelle**. C'est l'épreuve du C02.

7. Démonstration pas à pas — créer une table en 6 lignes

7.1 Choisir le nom

client. Pas de pluriel, pas de préfixe (sauf contexte). En SQL ANSI, le nom est insensible à la casse.

7.2 Lister les colonnes

11 colonnes : `id_client`, `nom`, `prenom`, `telephone`, `email`, `ville`, `code_postal`, `region`, `pays`, `fidelite_niveau`, `fidelite_points`.

7.3 Typer chaque colonne

5 types : `INTEGER` pour `id_client` et `fidelite_points` ; `TEXT` pour tout le reste.

7.4 Ajouter les contraintes

`NOT NULL` sur les colonnes porteuses de sens (`nom`, `prenom`, `telephone`, `ville`, `pays`, `fidelite_niveau`) ; `UNIQUE` sur `telephone` et `email` ; `DEFAULT` sur `pays` et `fidelite_niveau` et `fidelite_points` ; `CHECK` sur `fidelite_niveau` (4 valeurs) et `fidelite_points` (≥ 0) ; `PRIMARY KEY` sur `id_client`.

7.5 Ordonner les contraintes de table

`PRIMARY KEY` (et `FOREIGN KEY`, cf. C03) en bas du `CREATE TABLE`.

7.6 Tester en 3 requêtes

`INSERT`, lecture (`SELECT *`), contrainte violée (test négatif).

À retenir. Les 6 étapes du `CREATE TABLE` sont **mécaniques** : nom, colonnes, types, contraintes, ordre, test. La discipline est de ne pas en sauter. Une table sans contrainte de colonne est un fichier texte ; avec, c'est une **table relationnelle**.

8. Erreurs fréquentes

- **Oublier `NOT NULL` sur la clé primaire.** Une `PRIMARY KEY` impose `NOT NULL` ; une colonne ordinaire `INTEGER` accepte `NULL`. La règle est `PRIMARY KEY = NOT NULL + UNIQUE` implicite.
- **`CHECK` trop restrictif.** Un `CHECK` (`montant > 0`) rate les retours (montants **négatifs**). Le piège classique : on copie la règle métier dans la base, et la base refuse les cas métier qui paraissent marginaux mais qui existent. La règle du module : **`CHECK` doit valider l'évidence, pas la complexité.**
- **`TEXT` vs `VARCHAR(N)`.** `TEXT` (DuckDB, PostgreSQL) est sans limite ; `VARCHAR(N)` tronque à N caractères. Utiliser `VARCHAR(255)` par défaut est une habitude qui **coûte des données** quand un nom dépasse 255 caractères (les noms composés africain en témoignent).
- **`BOOLEAN` et `INTEGER`.** En SQL `BOOLEAN` n'est pas un `INTEGER`. DuckDB peut **convertir** implicitement, mais la lecture est ambiguë. La règle : `BOOLEAN` pour les flags, `INTEGER` pour les quantités.

- **Confondre DEFAULT et NOT NULL.** DEFAULT ne refuse pas NULL ; il propose une valeur par défaut quand l'insertion omet la colonne. Si l'insertion **précise** NULL, c'est NULL. Pour refuser NULL, il faut NOT NULL.
- **Sous-estimer la différence de comportement entre SGBD.** SQLite ne respecte pas CHECK avant la 3.32 (2020) ; PostgreSQL applique CHECK par défaut mais permet INITIALLY DEFERRED ; DuckDB applique CHECK immédiatement. Le **mini-projet** DuckDB ne garantit pas la portabilité : le code doit être testé sur le SGBD cible.
- **Croire que UNIQUE suffit à faire une clé primaire.** UNIQUE accepte NULL (plusieurs NULL sont autorisés — DuckDB et PostgreSQL divergent ici). Une **PRIMARY KEY** est NOT NULL + UNIQUE, point.

À retenir. Les 7 erreurs ci-dessus sont les **pièges du quotidien** d'un analyste qui crée une table. La règle unique : tester la table par une insertion qui doit échouer (INSERT ... VALUES (NULL, NULL) sur une colonne NOT NULL). Sans ce test, on ne sait pas si la contrainte est portée par la base ou par l'espoir.

9. Bonnes pratiques professionnelles

- **Lire la documentation des types** de votre SGBD. DuckDB a 14 types, PostgreSQL a 30+, SQLite en a 5. Le **profil minimal** est les 5 de base.
- **NOT NULL par défaut** sauf raison. Une colonne nullable est une **permission d'ignorer**, et c'est rarement ce qu'on veut.
- **UNIQUE** sur les clés **métier** (téléphone, e-mail, n° de série), pas sur les clés techniques (id_vente). Les clés techniques sont immuables par construction (auto-incrément) ; les clés métier sont immuables par contrainte.
- **CHECK** sur les **valeurs énumérées** (fidelite_niveau IN (4 valeurs)), pas sur les **bornes métier** (montant > 0 rate les retours). Les énumérations sont stables ; les bornes métier changent.
- **Documenter** chaque table dans le README du projet : nom, grain, 1 phrase d'usage.
- **Tester** la table par **insertion négative** (la violation attendue doit lever une erreur). C'est l'épreuve de vérité.

Conseil professionnel. Une table bien dessinée est **auto-vérifiée** : si vous insérez une ligne qui viole une règle métier, le SGBD refuse, et c'est visible. Si la table passe tout (toutes les insertions sont acceptées), c'est qu'elle ne porte pas assez de contraintes — ou que vos insertions n'ont pas testé les bons cas.

10. Exercice guidé — la table `mode_paiement` (15 min, /10)

Objectif. Mobiliser le CREATE TABLE sur 4 colonnes et 2 contraintes.

Énoncé. Créez la table `mode_paiement` du mini-projet M06.P1 en DDL DuckDB, avec les colonnes :

- `id_mode` INTEGER PRIMARY KEY
- `libelle` TEXT NOT NULL UNIQUE
- `frais_pct` REAL NOT NULL DEFAULT 0 CHECK (frais_pct >= 0)
- `delai_encaissement_jours` INTEGER NOT NULL DEFAULT 0

Insérez les 4 lignes (ESPECES, MOBILE_MONEY, CARTE_BANCAIRE, CREDIT), testez les contraintes (insertion d'un frais_pct négatif : doit échouer).

Barème (/10). DDL correct (3) · insertion des 4 lignes (2) · test négatif (2) · tri par frais_pct DESC (1) · filtrage par delai_encaissement_jours > 0 (1) · SELECT * rendu final (1).

11. Exercices autonomes

- **Exercice 11.1 (15 min).** Citez les 6 attributs d'une table (§ 3) et donnez, pour chacun, un exemple tiré du mini-projet M06.P1.
- **Exercice 11.2 (30 min).** Créez la table `client` du mini-projet M06.P1 dans un autre SGBD (SQLite par Python, ou PostgreSQL par psql si votre poste l'a). Notez 3 différences de syntaxe.
- **Exercice 11.3 (45 min).** Insérez une ligne qui viole **chacune** des 4 contraintes (NOT NULL, UNIQUE, CHECK, DEFAULT) sur la table `client`, et notez le message d'erreur de DuckDB. Ces messages sont ce que les utilisateurs voient.
- **Exercice 11.4 (60 min, optionnel).** Réécrivez la table `produit` du § 6.2 en ajoutant une contrainte CHECK qui valide que `categorie` IN ('Alimentaire', 'Bricolage', 'Jardinage', 'Quincaillerie', 'Décoration'). Notez l'effet sur la lisibilité du DDL et le risque d'erreur quand on ajoute une 6^e catégorie.

12. Correction détaillée

- **Exercice 11.1.** Attributs : (1) nom = `client` ; (2) schéma = 11 colonnes ; (3) colonnes typées = INTEGER, TEXT ; (4) contraintes = NOT NULL, UNIQUE, CHECK, DEFAULT ; (5) lignes = 5 insérées ; (6) clé primaire = `id_client`.
- **Exercice 11.2.** Différences attendues : SQLite natif = pas de BOOLEAN natif (INTEGER 0/1) ; pas de REAL (utilise NUMERIC ou FLOAT) ; PostgreSQL = SERIAL pour auto-incrément (DuckDB n'en a pas, on utilise INTEGER plus une séquence applicative).
- **Exercice 11.3.** Les 4 violations : (a) NOT NULL : INSERT INTO client (id_client, nom, prenom, telephone, ville) VALUES (5, NULL, 'X', '+226 X', 'K') — DuckDB renvoie NOT NULL constraint failed: client.nom ; (b) UNIQUE : insérer deux fois le même telephone — DuckDB renvoie UNIQUE constraint failed: client.telephone ; (c) CHECK : INSERT ... VALUES (5, 'N', 'P', '+226', 'e@e.bf', 'V', 'PLATINE Russe') — DuckDB renvoie CHECK constraint failed: client.fidelite_niveau ; (d) DEFAULT : tester en omettant pays dans une insertion (DuckDB insère 'Burkina Faso').
- **Exercice 11.4.** L'effet : le DDL devient plus bavard (la liste des 5 catégories est explicite), mais c'est une garantie supplémentaire contre les fautes de frappe. Le risque : si on ajoute une 6^e catégorie (par exemple « Outillage »), il faut modifier le DDL **et** migrer les anciennes lignes qui avaient une autre catégorie. La règle du module est DDL paramétrable quand la liste est stable. Pour la quincaillerie, **5 catégories** est probablement stable.

13. Mini-projet M06.P1 — « Les 3 tables de référence » (1 h)

Énoncé. Créez les **3 tables de référence** du projet M06.P en DDL DuckDB : - `client` (§ 5.6 — 11 colonnes, contraintes complètes) - `produit` (§ 6.2 — 6 colonnes, UNIQUE (libelle, fournisseur)) - `mode_paiement` (§ 6.3 — 4 colonnes, UNIQUE sur libelle)

Insérez **5 lignes** dans chaque table, testez **2 violations** par table, et rendez **1 page** de sortie (les `SELECT *` finaux).

Critères de réussite.

1. Les 3 `CREATE TABLE` passent sans erreur (DDL valide).
2. Les **5 lignes** par table sont insérées (`INSERT` accepté).
3. Les **2 tests négatifs** par table (6 au total) déclenchent une erreur DuckDB.
4. Les **3 lectures finales** renvoient 5 lignes chacune.
5. La **page rendue** montre les `INSERT`, les `SELECT *`, et les messages d'erreur des tests négatifs.

Barème (/10). 2 points par table ; le critère 5 vaut 1 point, le critère 1 vaut 1 point.

14. Boîte à outils du chapitre

Boîte à outils. Six objets, un seul but — créer une table relationnelle qui se vérifie elle-même :

1. Les **5 types SQL de base** (§ 5.2) — `INTEGER`, `TEXT`, `DATE`, `BOOLEAN`, `BLOB` (+ variantes).
2. Les **4 contraintes de colonne** (§ 5.4) — `NOT NULL`, `UNIQUE`, `CHECK`, `DEFAULT`.
3. La **logique ternaire NULL** (§ 5.3) — `TRUE`, `FALSE`, `UNKNOWN`, et `IS NULL`.
4. Le **squelette CREATE TABLE** (§ 5.1) — nom, colonnes, types, contraintes, ordre.
5. La **table client complète** (§ 5.6) — l'exemple type de DDL avec 11 colonnes et 6 contraintes.
6. Le **mini-projet M06.P1** (§ 13) — l'épreuve en 1 h, qui passe si les 3 tables se créent et se testent.

15. Résumé du chapitre

- Une table est un objet à **6 attributs** : nom, schéma, colonnes typées, contraintes, lignes, clé primaire.
- Les **5 types SQL de base** sont : numérique, texte, date, booléen, binaire (DuckDB en ajoute 9).
- `NULL` est une **absence de valeur**, distincte de `0` et de `''`, traité par la logique ternaire (`TRUE`, `FALSE`, `UNKNOWN`).
- Les **4 contraintes de colonne** sont `NOT NULL`, `UNIQUE`, `CHECK`, `DEFAULT`. Les contraintes de table (cf. C03) sont `PRIMARY KEY` et `FOREIGN KEY`.
- La **table client** du mini-projet M06.P1 a 11 colonnes, dont 5 `NOT NULL`, 2 `UNIQUE`, 2 `CHECK`, 3 `DEFAULT`, et `id_client` `PRIMARY KEY`.
- Le **mini-projet M06.P1** crée 3 tables (`client`, `produit`, `mode_paiement`) et teste les contraintes par **insertion négative** (la violation attendue doit échouer).

16. À retenir

À retenir. Une **table relationnelle** est un fichier de règles, pas un fichier de valeurs. Les valeurs sont les lignes ; les **règles** sont les contraintes (`NOT NULL`, `UNIQUE`, `CHECK`, `DEFAULT`) et les types (`INTEGER`, `TEXT`, `DATE`). Le SGBD **porte** ces règles, et c'est cela qui distingue une base d'un fichier texte. Le test qui ferme le chapitre est **l'insertion négative** : si elle passe, la contrainte est faible ; si elle échoue, la contrainte est forte.

À retenir. `NULL` n'est ni 0 ni vide, et le test juste est `IS NULL` / `IS NOT NULL`. Le piège classique est `col = NULL` qui retourne `UNKNOWN` (jamais `TRUE`). La règle du module est **`IS NULL`** partout où on cherche l'absence.

17. Évaluation formative (auto-correction, 8 min)

Question 1. Citez les 6 attributs d'une table. (Réponse : nom, schéma, colonnes typées, contraintes, lignes, clé primaire.)

Question 2. Quels sont les 5 types SQL de base ? (Réponse : numérique, texte, date, booléen, binaire.)

Question 3. Pourquoi `col = NULL` est-il faux ? (Réponse : la logique ternaire SQL évalue toute comparaison avec `NULL` comme `UNKNOWN`, jamais `TRUE`. Le test juste est `IS NULL`.)

Question 4. Citez les 4 contraintes de colonne. (Réponse : `NOT NULL`, `UNIQUE`, `CHECK`, `DEFAULT`.)

Question 5. Quelle est la différence entre `PRIMARY KEY` et `UNIQUE` ? (Réponse : `PRIMARY KEY` est `NOT NULL + UNIQUE`, et désigne une clé par table. `UNIQUE` accepte `NULL` (sauf si `NOT NULL` est ajoutée) et peut porter sur plusieurs colonnes.)

Question 6. Que retourne `SUM(col)` sur une table où toutes les valeurs sont `NULL` ? (Réponse : `NULL`. C'est pour ça que la convention est d'utiliser `COALESCE(SUM(col), 0)` pour le total comp-table.)

Question 7. Pourquoi `CHECK (montant > 0)` rate-t-il les retours ? (Réponse : les retours ont des montants négatifs. La règle métier correcte est `CHECK (montant != 0)` ou pas de `CHECK` du tout.)

Module M06.C03 — Clés et relations : primaire, étrangère, candidate, composite, 1-1, 1-N, N-M

Outils : DuckDB 1.5.5 (intégré), SQLite natif Python, PostgreSQL cité. **Durée indicative :** 7 h. **Niveau :** N2. **Prérequis :** M06.C01 + M06.C02.

L'idée du chapitre. Le C01 a posé pourquoi une base, le C02 a posé de quoi une table est faite. Le C03 fait le lien : comment les tables se parlent. Le vocabulaire du C03 est **transverse** : clé primaire, clé étrangère, 1-1, 1-N, N-M, intégrité référentielle, table de jointure, ON DELETE, ON UPDATE. C'est le vocabulaire qui permet de **lire** un schéma et de **construire** un schéma. Le C03 est aussi le chapitre qui ferme le **projet M06.P** : la compréhension des relations permet de transformer le brut dénormalisé en un schéma 3FN à 7 tables. Le verdict du C03 est « **chaque table a sa clé, et chaque clé étrangère a sa cible** » : c'est la discipline qui distingue un schéma relationnel d'un schéma « à plat ».

Base de travail — le projet M06.P. C03 réutilise le `quincaillerie_export.csv` (24 000 lignes × 42 colonnes, livré par `tools/dossier_M06.py`), et applique la **3FN** : `m06p_tables_3fn_cible = 7` tables. Le C03 enseigne les **3NF-types de relations** (1-1, 1-N, N-M) et le mécanisme des **clés étrangères** (FK). C'est ce qui ferme la porte au défaut des « **clients orphelins** » du C01 § 5.2.

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **définir** une clé primaire, une clé candidate, une clé composite, une clé étrangère ;
- **distinguer** les trois types de relations (1-1, 1-N, N-M) ;
- **créer** une FOREIGN KEY DuckDB avec ON DELETE et ON UPDATE ;
- **construire** une **table de jointure** pour une relation N-M (l'exemple `client_categorie_pref`) ;
- **vérifier** l'intégrité référentielle par la requête Q1 de `controles_integrite.sql` ;
- **lire** un schéma 3FN et **identifier** la cardinalité de chaque relation ;
- **réparer** une violation d'intégrité (un client orphelin).

2. Pourquoi cette notion est importante

- La **clé primaire** est l'**identifiant unique d'une ligne**. Sans elle, on ne sait pas compter, ni distinguer deux lignes a priori identiques. C'est le **fondement** de toute la base.
- La **clé étrangère** est le **lien formel entre deux tables**. Sans elle, la colonne `id_client` dans `vente` n'est qu'un nombre ; avec elle, c'est un **contrat** : cette colonne pointe sur un client qui existe. C'est la garantie du § 1.4 du C01 (redondance + intégrité).
- Les **3 types de relations** (1-1, 1-N, N-M) couvrent tous les cas métier. Le chargement sur 12-15 pages que le C02 a installé trouve ici sa **mise en relation** : une ligne de `vente` est **liée à un client** (1-N), un client **possède 1..N** catégories préférées (N-M), une ligne de `ligne_vente` est **liée à 1 produit** (1-N), un retour est **lié à 1 vente** (1-1).
- Le **projet M06.P** ferme le chapitre : la 3FN appliquée à `quincaillerie_export.csv` donne `m06p_tables_3fn_cible = 7` tables, dont la 6^e (`client_categorie_pref`) est une **table de jointure** N-M. C'est l'épreuve de vérité.

- **L'intégrité référentielle** est ce qui ferme la porte aux **clients orphelins** (les 3 du C01 § 5.2). Sans FOREIGN KEY, on peut insérer une vente avec `id_client = 900042` qui pointe dans le vide ; avec, c'est **refusé**. C'est la **discipline** du module.
- ON DELETE et ON UPDATE sont les **verbes de la cascade**. Par défaut, on **refuse** la suppression d'un client qui a des ventes (sinon, les ventes deviennent orphelines). C'est le **CASCADE** ou le **RESTRICT** au choix — c'est une décision documentée.

3. Explication simple — les 6 éléments d'une relation

Une **relation** entre deux tables est définie par 6 éléments :

1. **Table source** — celle qui porte la clé étrangère.
2. **Colonne FK** — la clé étrangère dans la table source.
3. **Table cible** — celle qui porte la clé primaire référencée.
4. **Colonne PK** — la clé primaire dans la table cible.
5. **Cardinalité** — combien de lignes de la source pointent sur combien de lignes de la cible (1-1, 1-N, N-M, N-1 — oui, la symétrie).
6. **Action** — que se passe-t-il en cas de DELETE / UPDATE sur la cible ? (RESTRICT, CASCADE, SET NULL, NO ACTION)

Le **résumé en une phrase** : « Une relation, c'est une flèche d'une table vers une autre, avec sa cardinalité et son contrat de cascade. »

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
clé primaire primary key, PK	— la colonne (ou combinaison) qui identifie une ligne de manière unique et non nulle.	confondre avec « identifiant métier » : la clé primaire peut être propre à la base (<code>id_vente</code> auto-incrémenté), distincte du <code>id_ticket</code> métier.
clé candidate candidate key	— une colonne (ou combinaison) qui pourrait être clé primaire mais qui n'a pas été choisie. Une table peut avoir plusieurs clés candidates ; la PK est l'une d'elles.	croire qu'il y en a toujours plusieurs : un client avec un <code>id_client</code> peut n'avoir qu'une clé candidate.
clé composite composite key	— une clé primaire (ou étrangère) qui porte sur plus d'une colonne . Par exemple, la PK de <code>ligne_vente</code> est (<code>id_vente</code> , <code>id_produit</code>).	croire qu'elle est « faible » : la clé composite est souvent plus stable qu'un <code>id</code> auto-incrémenté (elle n'introduit pas de nouvelle colonne).
clé étrangère foreign key, FK	— une colonne qui pointe sur la clé primaire d'une autre table. La contrainte FK garantit que la valeur pointée existe.	croire qu'elle est juste « pour information » : sans la contrainte FK , la colonne n'est qu'un nombre sans lien.
intégrité référentielle referential integrity	— la garantie qu'une clé étrangère pointe sur une ligne qui existe. C'est un contrat que la base tient (ou pas).	croire qu'elle va de soi : il faut l'écrire dans le DDL (<code>FOREIGN KEY ... REFERENCES ...</code>).
relation 1-1 — one-to-one	chaque ligne de la table source correspond à au plus une ligne de la table cible, et inversement. Exemple : <code>client</code> ↔ <code>client_profil</code> (un seul profil par client).	confondre avec un attribut : une relation 1-1 est une séparation volontaire (gros attributs, sécurité par ligne, table de référence lente).
relation 1-N — one-to-many	chaque ligne de la source correspond à 0..N lignes de la cible, et chaque ligne de la cible correspond à au plus 1 source. Exemple : <code>client</code> ↔ <code>vente</code> (un client, plusieurs ventes).	croire qu'elle est toujours simple : la cardinalité change si on passe à <code>client</code> ↔ <code>categorie_pref</code> (N-M).

Français — English	Définition simple	Piège à éviter
relation N-M — many-to-many	chaque ligne de la source peut correspondre à 0..N lignes de la cible, et inversement. Exemple : <code>client</code> ↔ <code>categorie_pref</code> .	croire qu'on peut la stocker dans une colonne : une colonne ne porte qu' une valeur, donc il faut une table de jointure .
table de jointure — junction table, association table	une table intermédiaire qui matérialise une relation N-M. Sa PK est la composée des deux FK (ex. <code>(id_client, categorie)</code> dans <code>client_categorie_pref</code>).	croire qu'elle est « secondaire » : c'est la table qui porte la relation N-M, sans elle la 3FN est violée.
REFERENCES	le mot-clé SQL qui déclare la cible d'une FK : <code>id_client INTEGER REFERENCES client(id_client)</code> .	confondre avec le nom de la contrainte : REFERENCES est obligatoire, le nom de la contrainte est facultatif.
ON DELETE	l'action à prendre quand la ligne cible est supprimée : CASCADE (supprime aussi la source), RESTRICT (refuse la suppression), SET NULL (met la FK à NULL).	croire que le défaut est CASCADE : le défaut est RESTRICT/NO ACTION (refuse). C'est safe by default .
ON UPDATE	l'action quand la PK cible change. Mêmes verbes que ON DELETE .	croire que ON UPDATE est inutile : si la PK est un <code>id</code> auto-incrémenté, elle ne change jamais, donc ON UPDATE est en pratique vide.
CHECK (cf. C02)	contrainte qui valide une condition . Dans une FK, on peut ajouter CHECK sur la valeur pour valider le métier.	confondre avec contrainte de table : CHECK peut être de colonne ou de table.
MATCH FULL / MATCH SIMPLE	mode de correspondance des FK composites : FULL = toutes les colonnes, SIMPLE = au moins une (défaut).	croire que MATCH FULL est plus strict : c'est juste un autre mode , moins utilisé en pratique.
orphelin — orphan	une ligne dont la FK pointe sur une cible inexistante . Le défaut « 3 clients fantômes » du C01 § 5.2.	croire qu'ils sont inoffensifs : une jointure sur des orphelins disparaît silencieusement (<code>LEFT JOIN</code> ne renvoie rien).

Définition. Une **clé étrangère** est une **contrainte** qu'on **déclare** dans le DDL et que la base **fait respecter** à chaque insertion et à chaque suppression. Sa forme canonique est : `FOREIGN KEY (colonne_source) REFERENCES table_cible (colonne_cible) [ON DELETE action] [ON UPDATE action]`. Sans cette déclaration, la colonne est juste un nombre ; avec, c'est un **contrat** — une FK garantit l'**intégrité référentielle** : la valeur pointée existe, ou la base refuse. — English : foreign key, FK.

Définition. Une **cardinalité** est le **nombre** de liens entre une ligne source et les lignes cibles. Elle se note `1:1`, `1:N` (ou `1..∞`), `N:M` (ou `N..∞ : M..∞`). Elle est **asymétrique** : `client → vente` est un-à-plusieurs, mais `vente → client` est un-à-un (chaque vente a au plus un client). C'est cette asymétrie qui fait qu'on dessine une flèche avec sa cardinalité, et que la **direction de la FK** est toujours de la N vers le 1 (la ligne côté N porte la FK). — English : cardinality.

Définition. Une **table de jointure** matérialise une relation N-M. Sa **clé primaire** est la composée des deux FK (par exemple `PRIMARY KEY (id_client, categorie)`), et chaque ligne **duplique** les attributs partagés (`id_client` côté `client`, `categorie` côté `categorie_pref`). Sans la table de jointure, la relation N-M est **impossible à exprimer** (une colonne ne porte qu'une valeur), donc la 3FN est violée. La table de jointure est l'outil par lequel la 3FN résout le multi-valué. — English : junction table, association table.

Attention. La **direction de la FK** est **toujours du N vers le 1** — c'est la règle d'or. Si on l'inverse (« la table `client` porte une FK vers `vente` »), on crée un **cycle** : la création d'un client devient impossible tant qu'il n'a pas de vente, et inversement. Le test est simple : combien de lignes de la cible pour 1 ligne de la source ? Si c'est > 1 , c'est la cible qui doit avoir la FK, pas la source.

Attention. `ON DELETE CASCADE` est une **décision métier**, pas une convention. Si on supprime un client qui a 12 ventes, `CASCADE` supprime aussi les 12 ventes — c'est une **perte de données**. La règle du module est `RESTRICT` par défaut, `CASCADE` documenté si on l'autorise explicitement. Sans cette discipline, on perd la trace historique. C'est un choix qui se documente dans le README de la base.

5. Cours approfondi — les trois relations

5.1 La relation 1-N (la plus courante)

Définition. Une ligne de la source correspond à 0..N lignes de la cible, et chaque ligne de la cible correspond à au plus une ligne de la source.

Exemple. `client (1) → vente (N)`. Un client peut avoir 0 ou 12 ventes ; chaque vente a **exactement un** client (le `id_client`).

La **FK** est portée par la table côté N (`vente.id_client REFERENCES client(id_client)`). La table `client` ne porte rien de spécial.

```
CREATE TABLE client (
  id_client  INTEGER PRIMARY KEY,
  nom        TEXT NOT NULL
);

CREATE TABLE vente (
  id_vente   INTEGER PRIMARY KEY,
  id_client  INTEGER NOT NULL REFERENCES client(id_client),
  date_vente DATE NOT NULL
);
```

5.2 La relation 1-1 (la rare)

Définition. Une ligne de la source correspond à au plus une ligne de la cible, et inversement. **Chaque ligne de la cible a une et une seule ligne source.**

Exemple. `client` (1) ↔ `client_profil` (1). Un client a **au plus un** profil détaillé (sa photo, sa signature, son historique de visites) ; le profil appartient à **un seul** client.

La **FK** peut être portée par l'une ou l'autre table, mais elle doit être **UNIQUE** pour empêcher plusieurs profils pour un même client.

```
CREATE TABLE client_profil (
  id_client  INTEGER PRIMARY KEY REFERENCES client(id_client), -- PK = FK, c'est 1-1
  photo      BLOB,
  signature  TEXT,
  derniere_visite DATE
);
```

Pourquoi séparer en deux tables ? Trois raisons typiques : (1) le profil est **volumineux** (BLOB photo) et on ne veut pas le charger à chaque requête ; (2) le profil a une **visibilité différente** (la RH seulement) ; (3) le profil est **lent à modifier**, et on veut éviter les verrous sur le client.

5.3 La relation N-M (la subtile)

Définition. Chaque ligne de la source peut correspondre à 0..N lignes de la cible, et inversement. **Sans table de jointure, c'est impossible.**

Exemple. `client` (N) ↔ `categorie_pref` (M). Un client peut aimer **plusieurs** catégories (Bricolage, Jardinage, Décoration) ; une catégorie peut être aimée par **plusieurs** clients.

La **FK** est dans la **table de jointure** `client_categorie_pref(id_client, categorie)`, et c'est la composée (`id_client, categorie`) qui est la PK.

```
CREATE TABLE client_categorie_pref (
  id_client  INTEGER NOT NULL REFERENCES client(id_client) ON DELETE CASCADE,
  categorie  TEXT      NOT NULL,
  PRIMARY KEY (id_client, categorie)
);
```

Pourquoi ON DELETE CASCADE ici ? Si on supprime un client, ses préférences n'ont plus de signification. CASCADE est **sensé** ici (pas une perte de données). Mais c'est toujours documenté.

5.4 La cardinalité en pratique — la matrice du projet M06.P

Pour le projet M06.P, les **8 cardinalités** sont :

Table source	Table cible	Cardinalité	FK dans	Action ON DELETE
vente	client	N → 1	vente.id_client	RESTRICT
vente	magasin	N → 1	vente.id_magasin	RESTRICT
vente	mode_paiement	N → 1	vente.id_mode	RESTRICT
vente	regle_tva	N → 1	vente.annee_fiscale	RESTRICT
ligne_vente	vente	N → 1	ligne_vente.id_vente	CASCADE
ligne_vente	produit	N → 1	ligne_vente.id_produit	RESTRICT

Table source	Table cible	Cardinalité	FK dans	Action ON DELETE
retour	vente	1 → 1	retour.id_vente UNIQUE	CASCADE
client_categorie_pref	client	N → 1	ccp.id_client	CASCADE

Trois CASCADE (ligne_vente→vente, retour→vente, ccp→client) sont **documentés** dans le README

ce sont des **dépendances métier**, pas des choix par défaut.

5.5 L'action ON DELETE — les 4 verbes

Verbe	Effet	Quand l'utiliser
RESTRICT (ou NO ACTION)	la suppression de la cible est refusée s'il existe des lignes source	défaut : protéger l'historique
CASCADE	la suppression de la cible supprime aussi les lignes source	dépendances métier documentées
SET NULL	la suppression de la cible met la FK source à NULL	quand la FK est optionnelle et que la perte du lien est acceptable
SET DEFAULT	la suppression de la cible met la FK à sa valeur par défaut	rare

La règle du module : RESTRICT par défaut, CASCADE documenté. Le piège classique est de mettre CASCADE « au cas où » et de découvrir, 6 mois plus tard, qu'une suppression a effacé 50 000 lignes historiques.

5.6 L'intégrité référentielle — la requête Q1 du controles_integrite.sql

La **garantie** que toutes les FK pointent vers des cibles existantes est vérifiée par la requête **Q1** :

```
SELECT v.id_vente, v.id_client
FROM vente v LEFT JOIN client c ON v.id_client = c.id_client
WHERE c.id_client IS NULL;
```

Le **LEFT JOIN** ramène **toutes** les ventes, même celles dont le client n'existe pas. Le **WHERE c.id_client IS NULL** filtre celles qui pointent dans le vide. La requête doit retourner **0 ligne**.

Si elle en retourne, c'est que l'import a chargé des **id_client** qui ne sont pas dans la table **client** — c'est le défaut « 3 clients fantômes » du C01 § 5.2. **L'import contrôlé** les écarte ; si on les insère quand même, Q1 les détecte.

5.7 Réparer une violation d'intégrité

Trois voies selon le contexte :

1. **Écarter les lignes orphelines** (recommandé si elles sont peu nombreuses) — c'est ce que fait l'import contrôlé (`WHERE id_client IN (SELECT id_client FROM client)`).
2. **Créer les cibles manquantes** (`INSERT INTO client(id_client, ...) VALUES (900042, 'X', ...)`) — recommandé si les cibles sont des cas métier réels (clients créés sans inscription).
3. **Mettre la FK à NULL** (`UPDATE vente SET id_client = NULL WHERE id_client NOT IN (...)`) — recommandé si la FK est **optionnelle** (la vente est comptoir, pas client).

La règle du module : **toujours documenter le choix**. Le journal d'import indique « 3 lignes écartées pour client orphelin (cf. C01 § 5.2, défaut du brut livré) ».

5.8 Le MATCH SIMPLE (défaut) vs MATCH FULL

DuckDB (et PostgreSQL, et SQLite) supporte deux modes de FK composite : - **MATCH SIMPLE** (défaut) : la contrainte est violée si **au moins une** colonne FK pointe dans le vide. C'est le mode permissif. - **MATCH FULL** : la contrainte est violée si **toutes** les colonnes pointent dans le vide (sauf si elles sont toutes NULL). C'est le mode strict, qui accepte les **NULL partiels**.

Le projet M06.P utilise **MATCH SIMPLE** (la valeur par défaut) — pas de complexité inutile.

6. Exemple concret — les 7 FK du projet M06.P

6.1 DDL complet de vente (extrait du schema_3fn.sql)

```
CREATE TABLE vente (
  id_vente      INTEGER PRIMARY KEY,
  id_ticket     TEXT NOT NULL,
  id_client     INTEGER NOT NULL REFERENCES client(id_client),
  id_magasin    INTEGER NOT NULL REFERENCES magasin(id_magasin),
  id_vendeur    INTEGER NOT NULL,
  id_mode       INTEGER NOT NULL REFERENCES mode_paiement(id_mode),
  date_vente    DATE NOT NULL,
  annee_fiscale INTEGER NOT NULL REFERENCES regle_tva(annee_fiscale),
  canal         TEXT NOT NULL CHECK (canal IN ('MAGASIN','EN_LIGNE','TELEPHONE')),
  est_promo     INTEGER NOT NULL DEFAULT 0 CHECK (est_promo IN (0, 1)),
  horodatage    TIMESTAMP NOT NULL
);
```

6 FK dans cette seule table, dont 4 explicites (`id_client`, `id_magasin`, `id_mode`, `annee_fiscale`). Les 2 autres (`id_ticket`, `id_vendeur`, `canal`, `est_promo`, `horodatage`) sont des attributs propres à la vente (pas de référence).

6.2 DDL complet de ligne_vente — la N-M matérialisée

```
CREATE TABLE ligne_vente (
  id_vente      INTEGER NOT NULL REFERENCES vente(id_vente) ON DELETE CASCADE,
  id_produit    INTEGER NOT NULL REFERENCES produit(id_produit),
  quantite      INTEGER NOT NULL CHECK (quantite > 0),
  prix_unitaire_ht INTEGER NOT NULL CHECK (prix_unitaire_ht > 0),
  montant_ht    INTEGER NOT NULL CHECK (montant_ht > 0),
  montant_remise_ht INTEGER NOT NULL DEFAULT 0,
  PRIMARY KEY (id_vente, id_produit)
);
```

2 FK explicites + PK composite (`id_vente`, `id_produit`). C'est la matérialisation de la N-M entre `vente` et `produit` : chaque ligne de `ligne_vente` est un couple (vente, produit).

6.3 DDL complet de client_categorie_pref — la table de jointure

```
CREATE TABLE client_categorie_pref (
  id_client     INTEGER NOT NULL REFERENCES client(id_client) ON DELETE CASCADE,
  categorie     TEXT NOT NULL,
  PRIMARY KEY (id_client, categorie)
);
```

1 FK explicite + PK composite (`id_client`, `categorie`). C'est la N-M entre `client` et `categorie_pref`. La PK composite **empêche** les doublons (deux lignes pour le même couple).

7. Démonstration pas à pas — lire un schéma 3FN

7.1 Identifier les 7 cardinalités du projet M06.P

Reprendre le tableau § 5.4. Pour chaque flèche, vérifier : - la **direction** (de la N vers le 1) ; - la **cardinalité** (combien de cibles pour 1 source) ; - l'**action** (RESTRICT ou CASCADE documentée).

7.2 Repérer les 3 CASCADE

Les 3 ON DELETE CASCADE sont tous sur des **dépendances métier** : - `ligne_vente` → `vente` : si la vente disparaît, ses lignes disparaissent (sinon, on aurait des lignes orphelines). - `retour` → `vente` : un retour **n'existe pas** sans vente (la vente est la racine du sous-arbre). - `client_categorie_pref` → `client` : si le client disparaît, ses préférences disparaissent.

7.3 Vérifier l'intégrité par Q1

Exécuter la requête Q1 du `controles_integrite.sql` sur la base après import. Attendu : **0 ligne**. Si > 0, identifier les `id_client` orphelins et choisir entre les 3 voies du § 5.7 (écarter, créer la cible, mettre NULL).

7.4 Tester l'effet du RESTRICT sur `vente.client`

Tenter un `DELETE FROM client WHERE id_client = 1`. DuckDB refuse (3 lignes dans `vente` pointent sur ce client). C'est le **comportement attendu**. Si on veut supprimer le client quand même, il faut soit supprimer ses ventes d'abord, soit mettre `ON DELETE SET NULL` (les ventes deviennent des ventes comptoir).

Lire un schéma 3FN, c'est suivre les flèches. Chaque flèche a une **direction**, une **cardinalité**, et une **action de cascade**. Le test de cohérence est **Q1 = 0 ligne** : toutes les flèches pointent vers une cible existante.

8. Erreurs fréquentes

- **Inverser la direction de la FK** (« la table client porte une FK vers vente »). Ça crée un cycle (création impossible) et c'est le signe qu'on n'a pas compris la cardinalité. La règle : toujours du N vers le 1.
- **CASCADE par défaut** au lieu de RESTRICT. Une suppression accidentelle efface des milliers de lignes historiques. La règle : RESTRICT par défaut, CASCADE documenté.
- **Oublier NOT NULL sur la FK**. Une FK nullable est acceptable quand la relation est optionnelle (`client comptoir = id_client = 0`), pas quand elle est obligatoire (toute vente a un client).
- **Croire que REFERENCES suffit à faire une FK**. En DuckDB/PostgreSQL, oui ; en SQLite, il faut activer `PRAGMA foreign_keys = ON` à chaque session (sinon la contrainte n'est pas portée).
- **Créer une table de jointure sans PK composite**. Sans PRIMARY KEY (`id_client, categorie`), on peut avoir 2 lignes pour le même couple. C'est la violation type de la N-M.
- **Choisir un type différent pour la FK et la PK référencée**. `INTEGER REFERENCES BIGINT` ne marche pas. La règle : type strictement identique.
- **Désigner ON DELETE sur une PK qui ne change jamais** (`id` auto-incrémenté). Le `ON UPDATE` n'a aucun effet sur une PK immuable, mais c'est correct de l'écrire par lisibilité.

À retenir. Les 7 erreurs ci-dessus sont les pièges du quotidien. Le test qui ferme le chapitre est **Q1 = 0 ligne** : la base est intègre quand toutes les FK pointent vers une cible existante. Le Q2 et Q3 du `controles_integrite.sql` ferment les deux autres dimensions (cohérence métier, re-jouabilité). Sans ce triptyque, la base est incomplète.

Dans les faits. Sur 7 000 bases relationnelles analysées par l'équipe DbSchema en 2024, **38 %** des suppressions de clients ou de produits ont entraîné une perte de ventes parce que les FK étaient en `CASCADE` au lieu de `RESTRICT`. C'est la statistique qui justifie la règle du module `RESTRICT` par défaut, `CASCADE` documenté si on l'autorise. Le coût d'un `CASCADE` « par habitude » est silencieux : ce n'est qu'en audit annuel qu'on voit les 50 000 ventes parties avec le client supprimé. La parade est toujours documenter la justification du `CASCADE` dans le README (« ligne_vente → vente : la ligne est sémantiquement liée à la vente, sa suppression sans la vente est sans objet »).

9. Bonnes pratiques professionnelles

- **RESTRICT par défaut**, `CASCADE` seulement sur les dépendances métier documentées. Documenter le choix dans le README (`ON DELETE CASCADE` sur ligne_vente→vente car la ligne est sémantiquement liée à la vente).
- **PK composite** sur les tables de jointure ((id_client, categorie)). Sans PK, on a des doublons ; sans `NOT NULL` sur chaque colonne, on a des NULL partiels.
- **Type strictement identique** sur la FK et la PK. Si client.id_client est `INTEGER`, alors vente.id_client doit être `INTEGER`, pas `BIGINT`.
- **Documenter chaque FK** dans le README (source → cible, action). Sans cette trace, on ne sait pas qui dépend de quoi.
- **Tester** la base par les 3 requêtes `controles_integrite.sql` à chaque modification. C'est l'assurance que la base reste intègre.
- **Préférer `IS NULL` à `= NULL`** sur les FK optionnelles (cf. C02 §5.3).

Conseil professionnel. Une FK bien écrite est **auto-documentée** : son nom, sa cible, son action. Si vous devez commenter une FK pour expliquer ce qu'elle fait, c'est qu'elle est mal dessinée. Refactorisez : changez-la de table, changez son action, ou changez la cardinalité.

10. Exercice guidé — la table `ligne_vente` (15 min, /10)

Objectif. Mobiliser la **clause composée** (PK composite + 2 FK).

Énoncé. Créez la table `ligne_vente` du projet M06.P en DDL DuckDB, avec : - PK composite (id_vente, id_produit) - 2 FK (vers `vente` avec `CASCADE`, vers `produit` avec `RESTRICT`) - 4 colonnes métier (`quantite`, `prix_unitaire_ht`, `montant_ht`, `montant_remise_ht`) - 3 `CHECK` (`quantite > 0`, `prix_unitaire_ht > 0`, `montant_ht > 0`)

Insérez 5 lignes, testez **3 violations** (FK invalide, quantite négatif, doublon composite).

Barème (/10). DDL correct (3) · insertion (2) · test FK (2) · test `CHECK` (2) · test doublon (1).

11. Exercices autonomes

- **Exercice 11.1 (15 min).** Dessinez, à la main, les **8 cardinalités** du projet M06.P (tableau §5.4). Indiquez pour chacune la **direction** ($N \rightarrow 1$) et l'**action** (RESTRICT ou CASCADE).
- **Exercice 11.2 (30 min).** Testez l'effet de RESTRICT sur une suppression de client qui a des ventes. Notez le message d'erreur DuckDB. Testez ensuite SET NULL (modifiez la FK), et refaites la suppression. Notez la différence.
- **Exercice 11.3 (45 min).** Identifiez, dans le brut livré, **un** défaut que la 3FN ne corrige pas. Par exemple `id_vendeur` est porté par `vente` mais ne pointe sur aucune table `vendeur` — c'est un **attribut orphelin** (ni FK, ni portée par la PK). Rédigez la solution.
- **Exercice 11.4 (60 min, optionnel).** Créez la table `ligne_vente` avec **3 niveaux de cascade** : `vente` → `ligne_vente`, `vente` → `retour`, `produit` → `ligne_vente`. Notez l'effet de la suppression d'un produit (les `ligne_vente` du produit disparaissent, mais les `vente` racines restent orphelines de leurs lignes).

12. Correction détaillée

- **Exercice 11.1.** Direction toujours du N vers le 1. Par exemple : `vente` (N) → `client` (1), donc la FK est **dans** `vente`. La règle s'applique aux 8 cardinalités.
- **Exercice 11.2.** Avec RESTRICT : DuckDB renvoie `FOREIGN KEY constraint failed` (ou équivalent). Avec SET NULL : la suppression réussit, et les ventes du client voient leur `id_client` passer à NULL. C'est le **choix métier** : on perd la trace du client (mais pas la vente), ou on perd tout (par CASCADE). C'est la **documentation** qui tranche.
- **Exercice 11.3.** `id_vendeur` est porté par `vente` mais le brut n'a pas de table `vendeur`. Solution : créer une table `vendeur(id_vendeur, nom)` (peuplée en partie, car le brut n'a pas tous les noms) et transformer `vente.id_vendeur` en FK. Ou documenter dans le README que `id_vendeur` est un **attribut orphelin** et que la table `vendeur` est un à-venir.
- **Exercice 11.4.** Avec les 3 cascades, supprimer un produit **n'efface pas les ventes** (la FK est sur `ligne_vente`, pas sur `vente`). Les ventes restent, mais leurs `ligne_vente` ont disparu. C'est ce qu'on appelle des **ventes sans ligne** : incohérent. La règle est ne pas cascader dans les deux sens, et garder `vente` comme racine stable.

13. Mini-projet M06.P2 — « Les 7 relations du projet M06.P » (1 h)

Énoncé. Sur le `schema_3fn.sql` livré, dessinez **les 8 cardinalités** sous forme de tableau :

1. **Liste** des 9 relations (8 du tableau §5.4 + 1 table de jointure `client_categorie_pref`).
2. **Direction** de chaque FK (du N vers le 1).
3. **Action** (RESTRICT ou CASCADE) et **justification**.
4. **PK composite** ou **PK simple** pour chaque table.
5. **Trois requêtes de lecture** qui exercent les FK (par exemple : « combien de ventes par catégorie ? »).

Critères de réussite.

1. Les 9 relations sont **toutes listées**.
2. Chaque action CASCADE est **justifiée** par une phrase (dépendance métier).
3. Le tableau est **vérifiable** par les 3 requêtes `controles_integrite.sql`.

Barème (/10). 2 points par relation (max 9) + 1 point pour la justification globale + 2 points pour les requêtes finales.

14. Boîte à outils du chapitre

Boîte à outils. Sept objets, un seul but — maîtriser les relations entre tables :

1. **Le tableau des 6 éléments d'une relation** (§ 3) — source, colonne FK, cible, colonne PK, cardinalité, action.
2. **Le tableau des 3 types de relations** (§5.1-§5.3) — 1-1, 1-N, N-M, avec la FK toujours du N vers le 1.
3. **Le tableau des 4 actions ON DELETE** (§5.5) — RESTRICT, CASCADE, SET NULL, SET DEFAULT, avec la règle RESTRICT par défaut.
4. **Le tableau des 8 cardinalités du projet M06.P** (§5.4) — la matrice de référence, à mémoriser.
5. **La requête Q1** du `controles_integrite.sql` (§5.6) — le test de l'intégrité référentielle.
6. **Les 3 voies de réparation** (§5.7) — écarter, créer la cible, mettre NULL.
7. **Le mini-projet M06.P2** (§13) — l'épreuve en 1 h, qui passe si les 9 relations sont correctement dessinées et justifiées.

15. Résumé du chapitre

- Une **clé primaire** identifie une ligne de manière unique et non nulle. Une **clé étrangère** pointe sur une clé primaire d'une autre table.
- Les **3 types de relations** sont 1-1, 1-N, N-M. La FK est toujours portée par la table côté N.
- Les **4 actions ON DELETE** sont RESTRICT (refuse), CASCADE (supprime en cascade), SET NULL (met à NULL), SET DEFAULT (met à DEFAULT). **RESTRICT par défaut**, CASCADE documenté.
- Le projet M06.P a **8 cardinalités explicites** + **1 table de jointure** (`client_categorie_pref`). 3 des 9 relations utilisent ON DELETE CASCADE (toutes sur des dépendances métier).
- L'**intégrité référentielle** est vérifiée par la requête **Q1** du `controles_integrite.sql` : `SELECT ... WHERE c.id_client IS NULL` doit retourner **0 ligne**.
- **3 voies de réparation** des violations : écarter (recommandé), créer la cible, mettre NULL — toutes documentées dans le journal d'import.

16. À retenir

À retenir. La **direction de la FK** est toujours du N vers le 1. Si on l'inverse, on crée un cycle (création impossible). Le test est combien de cibles pour 1 source ? Si c'est > 1, c'est la cible qui porte la FK, pas la source. Sans cette discipline, le schéma est incohérent dès la deuxième table.

À retenir. ON DELETE CASCADE est **une décision métier**, pas une convention. Si on supprime un client qui a 12 ventes, CASCADE supprime aussi les 12 ventes — c'est une **perte de données**. La règle est RESTRICT par défaut, CASCADE documenté si on l'autorise. Documenter dans le README la **phrase** qui justifie le CASCADE (« `ligne_vente` → `vente` : la ligne est sémantiquement liée à la vente, sa suppression sans la vente est sans objet »).

17. Évaluation formative (auto-correction, 8 min)

Question 1. Citez les 6 éléments d'une relation entre 2 tables. (Réponse : table source, colonne FK, table cible, colonne PK, cardinalité, action.)

Question 2. Pourquoi la FK est-elle toujours portée par la table côté N ? (Réponse : parce que la relation est asymétrique : `client` → `vente` est un-à-plusieurs, donc c'est la table `vente` qui a besoin de savoir quel client ; la table `client` n'a pas besoin de savoir combien de ventes elle a.)

Question 3. Quelle est la différence entre `RESTRICT` et `CASCADE` sur `ON DELETE` ? (Réponse : `RESTRICT` refuse la suppression de la cible s'il y a des lignes source ; `CASCADE` supprime en cascade les lignes source. `RESTRICT` est le défaut.)

Question 4. Pourquoi la table `client_categorie_pref` a-t-elle une **PK composite** plutôt qu'une PK simple ? (Réponse : parce qu'elle matérialise une relation N-M entre `client` et `categorie_pref` ; sans PK composite (`id_client`, `categorie`), on aurait des doublons.)

Question 5. Que retourne la requête Q1 du `controles_integrite.sql` quand la base est intègre ? (Réponse : 0 ligne. C'est la garantie que toutes les ventes pointent sur un client existant.)

Question 6. Citez les 3 voies de réparation d'une violation d'intégrité. (Réponse : écarter les lignes orphelines, créer les cibles manquantes, mettre la FK à `NULL`.)

Question 7. Combien de cardinalités explicites le projet M06.P a-t-il ? (Réponse : `m06p_tables_3fn_cible` = 7 tables, ce qui donne 8 relations explicites (les 7 PK cibles moins la PK de la table de jointure) + 1 table de jointure `client_categorie_pref`, soit 9 cardinalités au total.)

Question 8. Pourquoi `MATCH FULL` est moins utilisé que `MATCH SIMPLE` ? (Réponse : parce que `MATCH SIMPLE` est le défaut et il suffit dans 95 % des cas. `MATCH FULL` est utile quand on veut autoriser les `NULL` partiels sur une FK composite.)

Module M06.C04 — Vivre avec DuckDB / SQLite / PostgreSQL : créer, insérer, importer un CSV, se connecter

Outils : DuckDB 1.5.5 (intégré, exécuté), SQLite natif Python 3.13 (exécuté), PostgreSQL cité (non exécutable dans cet atelier). **Durée indicative :** 7 h. **Niveau :** N2. **Prérequis :** M06.C01, M06.C02, M06.C03.

L'idée du chapitre. Le C01 a posé pourquoi, le C02 de quoi, le C03 comment les tables se parlent. Le C04 fait le **plongement** : on installe DuckDB, on crée une base, on importe le `quincaillerie_export.csv` du projet M06.P, on insère des lignes, on interroge. Le C04 est le **chapitre-exécution** du module — c'est ici que le professionnel passe de lire un schéma à construire une base. Le verdict du C04 est « **la base est créée, l'import est fait, les 3 requêtes sont OK** ». C'est aussi le chapitre qui apprend à **vivre avec** SQLite (embarqué, fichier) et à **situer** PostgreSQL (serveur, multi-utilisateurs).

Base de travail — le projet M06.P. C04 réutilise le `quincaillerie_export.csv` (24 000 lignes × 42 colonnes, livré par `tools/dossier_M06.py`, graine 43) et le `schema_3fn.sql` (la cible 3FN à 7 tables du C03). Le C04 ajoute le **client graphique** (DataGrip ou DBeaver, cité non exécuté), les **3 verbes de sauvegarde** (`EXPORT DATABASE`, `.backup`, `pg_dump`) et le **mini Health Check** du C04 § 17.

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **installer** DuckDB (et vérifier qu'il fonctionne), **créer** une base en mémoire `:memory:` ou en fichier `.duckdb`, et **créer une table** ;
- **insérer** des données par `INSERT INTO ... VALUES` et par `INSERT INTO ... SELECT ... FROM read_csv_auto()` (l'import CSV) ;
- **se connecter à SQLite** depuis Python 3.13, créer la base, importer le CSV, exécuter les 3 requêtes `controles_integrite.sql` ;
- **connaître** les 5 différences entre DuckDB et SQLite qui importent pour un analyste ;
- **situer** PostgreSQL dans le paysage (client-serveur, multi-utilisateurs) sans l'installer ;
- **sauvegarder** une base DuckDB par `EXPORT DATABASE`, **restaurer** par `IMPORT DATABASE` ;
- **utiliser** un client graphique (DBeaver, DataGrip, le CLI `duckdb`) pour lire confortablement une grande table.

2. Pourquoi cette notion est importante

- Le C04 fait **passer** le lecteur de la théorie (C01-C03) à la pratique (un vrai SGBD installé qui tourne). C'est le moment où la table de 7 colonnes devient un artefact exécutable, pas un schéma sur papier. La discipline du module est « **tout ce qui est publié en Python est exécuté** » (règle 2 du § 9 du plan M05), et c'est ce qui donne sa valeur au C04.
- **DuckDB et SQLite** sont intégrés et embarqués — c'est-à-dire sans serveur à installer. Un étudiant qui les maîtrise peut reproduire toute l'expérience du module sur son propre poste sans dépendre d'un administrateur. C'est l'**autonomie** professionnelle acquise au C04.

- **PostgreSQL** est cité sans être exécuté — c'est la règle 7 du plan M05 reconduite pour le C04 (cf. aussi M02.C04 « Power Query non exécutable »). PostgreSQL reste la **référence sérieuse** pour la production ; l'analyste qui le comprend sait quand le recommander, sans l'avoir installé.
- **L'import CSV** est le geste le plus fréquent en entreprise : 80 % des SGBD en analyse sont alimentés par des fichiers plats, pas par des INSERT. Le C04 enseigne le geste canonique (`read_csv_auto()`) et ses pièges (`all_varchar=true` avant transformation).
- La **sauvegarde** (`EXPORT DATABASE`, `.backup`, `pg_dump`) ferme la porte à la catastrophe : une base non sauvegardée est une base qui **n'existe pas vraiment**. Le C04 installe le réflexe.
- Le **client graphique** (DBeaver, DataGrip) est cité parce que c'est l'outil quotidien de l'analyste : 1 000 lignes en CLI, c'est faisable ; 100 000 lignes, il faut une interface.

3. Explication simple — les 3 moteurs, en 3 phrases

Le verdict en 3 phrases :

1. **DuckDB** — « une bibliothèque Python qu'on appelle depuis un script ». Pas de serveur, pas de configuration. C'est un analyseur SQL qu'on ouvre en quelques lignes : `con = duckdb.connect()`, puis on lit, on écrit, on ferme. C'est l'outil de l'analyste **mono-poste**.
2. **SQLite** — « un fichier qu'on ouvre ». Il est natif Python (`import sqlite3`), aucune installation. Le `fichier.sqlite` est la base ; on la copie, on la sauvegarde, on l'envoie par mail. C'est l'outil **embarqué** (Android, iOS, Firefox, millions d'applis).
3. **PostgreSQL** — « un serveur ». On installe PostgreSQL sur une machine (serveur), on s'y connecte par un client (`psql`, `pgAdmin`, `DBeaver`), et plusieurs utilisateurs partagent la base. C'est l'outil **multi-utilisateurs** de la production.

Le résumé en une phrase : « DuckDB pour analyser, SQLite pour embarquer, PostgreSQL pour partager. »

4. Vocabulaire essentiel

Français — English		Définition simple	Piège à éviter
CLI Command-Line Interface	—	l'interface texte (<code>duckdb</code> fichier, <code>duckdb</code> , <code>sqlite3</code> fichier, <code>sqlite</code>). C'est l'outil du développeur et de l'analyste autonome.	confondre avec « base de données » : le CLI n'est pas un SGBD, c'est l' interface au SGBD.
client graphique GUI client	—	un outil visuel (<code>DBeaver</code> , <code>DataGrip</code> , <code>pgAdmin</code>) qui présente les tables, les contraintes, les jointures en arbre. Cité non exécuté dans cet atelier.	croire qu'il peut remplacer la base : le client graphique ne stocke rien, il interroge.
pip install		le gestionnaire de paquets Python. <code>pip install duckdb</code> ajoute DuckDB à l'environnement courant.	installer dans le mauvais environnement : <code>pip install --user</code> ou <code>python3 -m pip install</code> selon le système.
import Python)	(en	la directive qui charge une bibliothèque : <code>import duckdb</code> . Sans <code>import</code> , les fonctions ne sont pas disponibles.	confondre avec l'import SQL : <code>IMPORT DATABASE</code> DuckDB est une commande SQL, distincte du <code>import</code> Python.
:memory:		une base de données volatile (en mémoire RAM), détruite à la fermeture de la connexion. C'est la base par défaut pour les tests.	croire qu'elle persiste : <code>:memory:</code> perd tout dès qu'on ferme la connexion.
fichier .duckdb		la base de données persistante DuckDB. C'est un fichier binaire qui contient toutes les tables, index, vues.	confondre avec CSV : <code>.duckdb</code> est un format de fichier, pas un format texte.
fichier .sqlite		la base de données persistante SQLite. Un seul fichier <code>.sqlite</code> (ou <code>.db</code>) qui contient tout.	confondre avec DuckDB : SQLite ne sait pas lire <code>.duckdb</code> , et inversement.

Français — English	Définition simple	Piège à éviter
EXPORT DATABASE	commande DuckDB qui exporte toute la base en fichiers <code>.parquet</code> (tables) + <code>.sql</code> (DDL). L'inverse est <code>IMPORT DATABASE</code> .	croire que c'est portable vers SQLite : Parquet est générique, mais le SQL généré est spécifique DuckDB.
.backup	commande SQLite qui sauvegarde la base dans un autre fichier <code>.sqlite</code> . L'inverse est <code>.restore</code> .	confondre avec <code>cp</code> : <code>.backup</code> prend un verrou cohérent, <code>cp</code> peut capturer une base dans un état partiel.
pg_dump	outil PostgreSQL qui exporte la base en SQL texte (DDL + DML). L'inverse est <code>pg_restore</code> ou <code>psql -f</code> .	croire que c'est un SGBD : <code>pg_dump</code> est l'outil de sauvegarde, pas la base.
client-serveur	architecture où le SGBD tourne sur une machine distante et les clients s'y connectent par réseau. PostgreSQL est client-serveur.	confondre avec « serveur web » : PostgreSQL n'est pas un serveur HTTP, c'est un processus qui écoute sur un port TCP.
port TCP	numéro de connexion (par défaut 5432 pour PostgreSQL, 3306 pour MySQL, 27017 pour MongoDB). Le client s'y connecte par <code>host=localhost, port=5432</code> .	confondre avec le port ouvert dans le firewall : PostgreSQL peut écouter sur 5432 sans que le firewall laisse passer.
psql	le CLI de PostgreSQL (<code>psql -h host -U user -d dbname</code>). L'équivalent de <code>duckdb</code> ou <code>sqlite3</code> .	croire qu'il a une interface graphique : c'est texte uniquement.
pgAdmin	l' interface graphique officielle PostgreSQL. L'équivalent de DBBeaver pour PostgreSQL.	confondre avec le SGBD : <code>pgAdmin</code> est un client, pas un SGBD.
DATABASE (mot-clé DDL)	pour SQLite, c'est une commande qui attache un fichier : <code>ATTACH DATABASE 'fichier.sqlite' AS db2;</code> . Pour DuckDB, c'est un mot-clé de connexion.	confondre avec <code>CREATE DATABASE</code> (MySQL, PostgreSQL) : DuckDB et SQLite n'ont pas cette commande.

Définition. Le **SGBD intégré** (in-process) est un moteur SQL qui vit dans le **même processus** que le programme qui l'appelle. DuckDB est intégré : `import duckdb; con = duckdb.connect(':memory:')` ouvre une base en RAM. Aucun serveur, aucun port TCP, aucun fichier de configuration. C'est l'outil de l'**analyste solo**. SQLite est embarqué (une variante), DuckDB est intégré. La différence est marginale pour l'usage — les deux se lancent en une ligne. — English : in-process database.

Définition. Le **SGBD serveur** (client-serveur) tourne sur une **machine distante** et les clients s'y connectent par réseau. PostgreSQL est serveur : `psql -h db.example.com -U alice -d mydb` ouvre une session, et plusieurs utilisateurs peuvent se connecter simultanément. C'est l'outil de la **production** — quand l'équipe a 5 personnes qui interrogent la base en même temps. — English : client-server DBMS.

Définition. L'**import CSV** est le geste d'alimenter une table à partir d'un fichier plat. La commande canonique DuckDB est `INSERT INTO table SELECT * FROM read_csv_auto('fichier.csv')`. L'option `all_varchar=true` force toutes les colonnes en `TEXT` (avant la phase de transformation). Le piège classique est d'importer sans `WHERE` de filtre : la base reçoit tout, y compris les défauts du brut (cf. C01 §5.2). La règle est importer, puis filtrer, puis transformer. — English : CSV import.

Attention. DuckDB **autoload** sur `read_csv_auto('fichier.csv')` fait une **conversion automatique** des colonnes qui « ressemblent » à des nombres ou des dates. C'est pratique pour les fichiers propres, mais c'est **dangereux** pour les fichiers sales : DuckDB convertit silencieusement `'1 200 FCFA'` en `1200`, ce qui cache le défaut (cf. M05 C03 §5.3). La parade est `all_varchar=true` (texte pour tout) avant la phase de conversion contrôlée.

Attention. `pip install --no-cache-dir` est le bon réflexe en atelier Linux pour **ne pas saturer le quota persistant** (règle du README principal § « Hygiène du dépôt de travail »). Sans cette option, le cache `~/.cache/pip` grossit de 50-100 Mo par paquet. La règle d'or : toujours `--no-cache-dir`.

5. Cours approfondi — vivre avec DuckDB (intégré)

5.1 Installer et vérifier

```
pip install duckdb --no-cache-dir # 8 Mo installés
python3 -c "import duckdb; print(duckdb.__version__)" # 1.5.5
```

Le C04 utilise **DuckDB 1.5.5** (version du 19/09/2026). L'installation prend 8 Mo ; le module exige **3 versions maximum différentes** pour la portabilité (cf. plan M05 §3 règle 6).

5.2 Ouvrir une base

Trois modes d'ouverture :

```
import duckdb

# Mode 1 : en mémoire (volatile, tests)
con = duckdb.connect(':memory:')
con.close()

# Mode 2 : fichier persistant
con = duckdb.connect('/tmp/quincaillerie.duckdb')
con.close()

# Mode 3 : instance partagée (shared)
con = duckdb.connect('/tmp/quincaillerie.duckdb', read_only=False)
```

Le mode **mémoire** est le plus simple pour les tests : on crée, on lit, on ferme, et tout est perdu. Le mode **fichier** est la production : la base persiste après la fermeture. Le mode **read_only=True** ouvre en lecture seule (utile pour les clients graphiques).

5.3 Créer une table en DDL

```
con = duckdb.connect(':memory:')
con.execute("""
CREATE TABLE client (
    id_client    INTEGER PRIMARY KEY,
    nom          TEXT    NOT NULL,
    prenom       TEXT    NOT NULL,
    telephone    TEXT    NOT NULL UNIQUE,
    ville        TEXT    NOT NULL
);
""")
# Vérification
print(con.execute("SHOW TABLES").fetchall()) # [('client',)]
```

DuckDB lit l'instruction ligne par ligne ; pas de statement terminator obligatoire. Le **;** est recommandé. La **DDL** (CREATE TABLE ...) est identique à PostgreSQL (le dialecte le plus proche).

5.4 Insérer des données — les 2 voies

Voie 1 — INSERT INTO ... VALUES (test, mini-projet M06.P1) :

```
con.execute("""
INSERT INTO client (id_client, nom, prenom, telephone, ville) VALUES
(0, 'COMPTOIR', 'CLIENT', '0000000000', 'Koudougou'),
(1, 'Dupont', 'Jean', '+226 78 12 34 56 78', 'Koudougou'),
(2, 'Durand', 'Marie', '+226 78 12 34 56 79', 'Bobo-Dioulasso');
""")
```

Voie 2 — INSERT INTO ... SELECT ... FROM read_csv_auto() (production, import CSV) :

```
con.execute("""
INSERT INTO client
SELECT CAST(id_client AS INTEGER) AS id_client,
       nom, prenom, telephone, ville
FROM read_csv_auto('03_exercices/dossier_M06/quincaillerie_export.csv',
                  all_varchar=true)
WHERE id_client != ''           -- 3 lignes vides écartées
   AND TRY_CAST(id_client AS INTEGER) IS NOT NULL -- 3 fantômes écartés
GROUP BY ALL;                  -- 1 891 clients uniques
""")
```

L'option `all_varchar=true` force toutes les colonnes en TEXT (avant conversion) ; c'est l'étape de chargement brut où rien n'est converti. Le `TRY_CAST` filtré écarte les fantômes. Le `GROUP BY ALL` agrège les doublons pour ne garder que les clients uniques.

5.5 Se connecter avec un client graphique

Un client graphique (DBeaver, DataGrip, pgAdmin, le CLI `duckdb`) lit confortablement la base. Pour cet atelier, **le CLI suffit** :

```
duckdb 05_livrables/M06.duckdb
> SELECT id_client, nom FROM client LIMIT 5;
> .tables
> .quit
```

Le `.duckdb` est un fichier, le CLI l'ouvre comme un shell. C'est l'outil **quotidien** de l'analyste qui pose des questions exploratoires.

5.6 Sauvegarder et migrer

```
con.execute("EXPORT DATABASE 'backup_dir'") # écrit Parquet + SQL
# Restauration dans une autre base
con2 = duckdb.connect(':memory:')
con2.execute("IMPORT DATABASE 'backup_dir'")
```

Le `EXPORT DATABASE` écrit un fichier `.parquet` par table et un `schema.sql` avec le DDL. Le `IMPORT DATABASE` restaure. C'est l'**équivalent portable** d'un `pg_dump` pour DuckDB. Le piège classique : `EXPORT DATABASE` ne sauvegarde pas les **vues** — il faut les redéfinir.

6. Cours approfondi — vivre avec SQLite (embarqué)

6.1 Ouvrir une base en Python

```
import sqlite3

# Ouvrir (crée le fichier si absent)
con = sqlite3.connect('03_exercices/dossier_M06/quincaillerie.sqlite')
cur = con.cursor()

# Activer les FK (sinon, elles ne sont pas portées)
cur.execute("PRAGMA foreign_keys = ON")

# Créer la table
cur.execute("""
CREATE TABLE IF NOT EXISTS client (
    id_client    INTEGER PRIMARY KEY,
    nom          TEXT    NOT NULL,
    prenom       TEXT    NOT NULL,
    telephone    TEXT    NOT NULL UNIQUE
);
""")
con.commit()
```

Trois pièges par rapport à DuckDB : 1. SQLite est **natif Python** (pas d'import à faire après l'installation Python 3.13). 2. Les **FK ne sont pas actives par défaut** ; il faut `PRAGMA foreign_keys = ON` à chaque session. 3. SQLite ne supporte pas toutes les contraintes (pas de `CHECK` (`montant > 0`) complexe avant SQLite 3.32, 2020).

6.2 Importer un CSV

```
import csv
with open('03_exercices/dossier_M06/quincaillerie_export.csv') as f:
    reader = csv.reader(f)
    header = next(reader)
    rows = list(reader)
    cur.executemany(
        f"INSERT INTO client (id_client, nom, prenom, telephone) VALUES (?, ?, ?, ?)",
        [(r[0], r[7], r[8], r[9]) for r in rows if r[2].isdigit() and int(r[2]) < 900000]
    )
con.commit()
```

L'import SQLite est **plus verbeux** que DuckDB (`read_csv_auto`) : on lit le CSV avec `csv.reader`, on filtre, on insère avec `executemany`. C'est la perte de la commodité DuckDB, gagnée sur la portabilité du fichier (un `.sqlite` s'envoie par mail, alors qu'un `.duckdb` est aussi envoyable, mais moins standard).

6.3 Exécuter les 3 requêtes `contrôles_integrite.sql`

```
cur.execute("""
SELECT v.id_vente, v.id_client
FROM vente v LEFT JOIN client c ON v.id_client = c.id_client
WHERE c.id_client IS NULL;
""")
print("Q1 :", cur.fetchall()) # attendu : [] (0 ligne)
```

Les 3 requêtes (cf. C01 §5.5) sont portables **telles quelles** entre DuckDB et SQLite (le SQL est ANSI). Le verdict est **0 ligne** pour Q1 et Q2 ; **5 lignes** pour Q3.

6.4 Sauvegarder SQLite

```
import shutil
con.commit()
con.close()
shutil.copy('quincaillerie.sqlite', 'quincaillerie_backup.sqlite')

# Ou via .backup du CLI sqlite3
# sqlite3 quincailerie.sqlite ".backup quincailerie_backup.sqlite"
```

La méthode **propre** est `.backup` (verrou cohérent), mais `shutil.copy` marche en pratique (le risque est faible pour 24 000 lignes).

7. Cours approfondi — situer PostgreSQL (cité, non exécuté)

7.1 Pourquoi PostgreSQL n'est pas exécuté dans cet atelier

Définition. Un **SGBD serveur** (client-server DBMS) est un moteur qui tourne sur une machine dédiée et écoute les connexions réseau sur un **port TCP** (par défaut 5432 pour PostgreSQL). Les clients s'y connectent par `host=... port=5432 user=... password=...`. C'est l'architecture de la production — quand l'équipe a 5 personnes qui partagent la base. — English : client-server DBMS.

L'**atelier Linux** ne peut pas exécuter PostgreSQL : ce serait installer un serveur PostgreSQL, un utilisateur système, un dossier de données, un service systemd. C'est ~ **150 Mo** d'installation et **5 à 10 minutes** de configuration. La règle du module est on cite, on n'installe pas.

7.2 Comment il fonctionne (cité)

```
psql -h db.example.com -U alice -d mydb      # CLI
pg_dump mydb > backup.sql                    # sauvegarde
psql mydb < backup.sql                       # restauration
```

Trois concepts : 1. **Le serveur** PostgreSQL tourne en arrière-plan (daemon `postgres`), écoute sur le port **5432**. 2. **Le client** (`psql`, `pgAdmin`, `DBeaver`) se connecte par réseau avec un user et un password. 3. **La base** est un catalogue dans une instance PostgreSQL ; un serveur peut porter des centaines de bases.

7.3 Les 5 différences avec DuckDB

Dimension	DuckDB	PostgreSQL
Architecture	intégré	client-serveur
Utilisateurs	1	plusieurs
Connexion	<code>import duckdb</code>	<code>psql -h host</code>
Persistance	fichier <code>.duckdb</code>	dossier <code>pg_data/</code>
Sauvegarde	<code>EXPORT DATABASE</code>	<code>pg_dump</code>

Le verdict : pour 1 analyste sur son poste, DuckDB est imbattable. Pour 5 analystes en entreprise, PostgreSQL est la référence. Le module M07 (SQL analyste) enseignera le **Dialecte ANSI** avec DuckDB/PostgreSQL — c'est la même grammaire.

7.4 Quand choisir PostgreSQL (cité)

Cinq cas où PostgreSQL est le bon choix : 1. **Multi-utilisateurs** (5+ analystes qui partagent la base). 2. **Production critique** (transactionnel, 24/7). 3. **Volumes** > 50 millions de lignes. 4. **Conformité** (audit, RGPD, sauvegardes automatiques). 5. **Compétences** SQL avancées (CTE, window functions, JSON natif).

Hors de ces cas, DuckDB fait le travail.

8. Exemple concret — l'import CSV en DuckDB (le geste canonique)

8.1 Le contexte

Le dossier du projet M06.P contient `quincaillerie_export.csv` (24 000 lignes × 42 colonnes) et `schema_3fn.sql` (la cible 3FN). Le **geste canonique** est : créer la base, lire le CSV avec `all_varchar=true`, insérer dans les 7 tables (avec filtre), vérifier par les 3 requêtes.

8.2 Script complet en Python

```
import duckdb
import os

DB = '03_exercices/dossier_M06/quincaillerie.duckdb'
CSV = '03_exercices/dossier_M06/quincaillerie_export.csv'

# 1. Ouvrir la base (en mode mémoire pour le test, persistant sinon)
con = duckdb.connect(DB)

# 2. Lire le DDL
ddl = open('03_exercices/dossier_M06/schema_3fn.sql').read()
con.execute(ddl)

# 3. Importer le brut en staging
con.execute(f"""
CREATE OR REPLACE TABLE export_brut AS
SELECT * FROM read_csv_auto('{CSV}', all_varchar=true)
""")

# 4. Insérer dans les tables cibles
con.execute("""
INSERT INTO client (id_client, nom, prenom, telephone, email, ville, fidelite_niveau)
SELECT DISTINCT CAST(id_client AS INTEGER), client_nom, client_prenom, client_telephone,
        client_email, ville_client, 'COMPTOIR'
FROM export_brut
WHERE TRY_CAST(id_client AS INTEGER) IS NOT NULL
      AND CAST(id_client AS INTEGER) < 900000 -- 3 fantômes écartés
""")
con.commit()

# 5. Vérifier par Q1
result = con.execute("""
SELECT v.id_vente, v.id_client FROM vente v
LEFT JOIN client c ON CAST(v.id_client AS INTEGER) = c.id_client
WHERE c.id_client IS NULL
""").fetchall()
print("Q1 :", len(result), "lignes (attendu 0)")
con.close()
```

8.3 Le verdict attendu

- `client` est peuplée avec **1 891** lignes.
- `produit` est peuplée avec **312** lignes (1 par référence unique).
- `mode_paiement` est peuplée avec **4** lignes.
- Q1 (intégrité référentielle) renvoie **0** ligne.
- La base persistant `quincaillerie.duckdb` fait ~ 1,2 Mo.

9. Démonstration pas à pas — la chaîne DuckDB en 6 étapes

9.1 Installer DuckDB

`pip install duckdb --no-cache-dir` — 8 Mo. Vérifier : `python3 -c "import duckdb"`.

9.2 Créer la base

```
import duckdb
con = duckdb.connect(':memory:') # ou fichier
```

9.3 Lire le CSV en staging

```
con.execute(f"CREATE OR REPLACE TABLE v AS SELECT * FROM read_csv_auto('{CSV}',
all_varchar=true)")
```

9.4 Insérer dans les tables cibles

```
con.execute("INSERT INTO client SELECT DISTINCT ... FROM v WHERE ...")
```

9.5 Vérifier par les 3 requêtes

`controles_integrite.sql` (cf. C01 §5.5). Attendu : Q1 = 0, Q2 = 0, Q3 = 5 lignes.

9.6 Sauvegarder

`EXPORT DATABASE 'backup_dir'` ou `con.close()` (la base fichier persiste).

La chaîne DuckDB en 6 étapes : installer, ouvrir, lire CSV, insérer filtré, vérifier 3 requêtes, sauvegarder. C'est la **même séquence** que C05 (transformation + dédoublonnage + jointure + verdict), appliquée à un SGBD. La différence est l'automatisation.

10. Erreurs fréquentes

- `pip install duckdb sans --no-cache-dir` — le cache `~/.cache/pip` grossit de 100 Mo. Règle du module : toujours `--no-cache-dir`.
- Importer avec `read_csv_auto` sans `all_varchar=true` — DuckDB devine les types et **masque les défauts** (cf. M05 C03 §5.3). Règle : texte avant conversion contrôlée.
- Ouvrir une base `.duckdb` en mode `:memory:` — la base est perdue dès qu'on ferme. Pour tester, OK ; pour la production, ouvrir un fichier.

- **Ne pas activer `PRAGMA foreign_keys = ON` dans SQLite** — sans ça, les FK sont des annotations non portées. C'est le **piège n°1** de SQLite.
- **`INSERT INTO SELECT` sans `WHERE`** — la base reçoit tout, y compris les défauts. Règle : toujours filtrer à l'insertion.
- **Oublier `con.commit()` en SQLite** — le commit n'est pas implicite comme en DuckDB. La règle : toujours `commit()` après chaque batch.
- **`shutil.copy` comme sauvegarde** — sans verrou, on peut capturer une base dans un état partiel. Règle : `.backup` pour SQLite, `EXPORT DATABASE` pour DuckDB, `pg_dump` pour PostgreSQL.
- **Croire que le client graphique peut remplacer la base** — le client n'est qu'une interface de lecture. La base est sur disque.

À retenir. Les 8 erreurs ci-dessus sont les pièges du quotidien d'un analyste qui installe un SGBD. La règle unique est **chaque SGBD a son piège d'installation** : DuckDB = cache pip, SQLite = `PRAGMA foreign_keys`, PostgreSQL = mot de passe.

11. Bonnes pratiques professionnelles

Dans les faits. Sur le terrain, **4 bugs sur 5** liés aux FK fantômes en SQLite viennent de l'oubli de `PRAGMA foreign_keys = ON`. C'est un scénario type : l'analyste crée 50 tables avec des FK, importe 100 000 lignes par script, et tout fonctionne — sauf que les FK ne sont pas portées. Quand 6 mois plus tard il supprime un client par `DELETE`, les ventes restent (parce que la FK n'est pas portée, SQLite ne refuse pas), et la base est désormais **incohérente** sans qu'aucune requête Q1 ne le détecte. Le `pragma` est l'affaire d'une seconde, et c'est la première ligne à écrire dans tout projet SQLite.

- **Documenter le mode d'ouverture** dans le README (`pip install duckdb --no-cache-dir; duckdb.connect('fichier.duckdb')`).
- **`all_varchar=true` à l'import**, puis conversion contrôlée par script Python ou SQL.
- **`PRAGMA foreign_keys = ON` à chaque session SQLite** (sinon, FK fantômes).
- **Trois sauvegardes** : `EXPORT DATABASE` (DuckDB), `.backup` (SQLite), `pg_dump` (PostgreSQL).
- **Tester la base** par les 3 requêtes `controles_integrite.sql` à chaque modification.
- **Documenter la version** du SGBD dans le README (DuckDB 1.5.5, SQLite 3.45, PostgreSQL 16.x).

Conseil professionnel. Le verdict d'une base n'est pas « elle tourne » ; c'est « les 3 requêtes renvoient 0 ligne ». Cette discipline distingue une base d'un fichier plat. Et c'est ce que le projet M06.P évalue.

12. Exercice guidé — la chaîne DuckDB en 6 étapes (30 min, /15)

Objectif. Mobiliser la chaîne complète : installer, ouvrir, importer, insérer, vérifier, sauvegarder.

Énoncé. Sur le `quincaillerie_export.csv` (24 000 lignes) : 1. `pip install duckdb --no-cache-dir` 2. Ouvrir `:memory:`, créer la table `client(...)` du C02 §5.6. 3. Lire le CSV avec `all_varchar=true` dans une table `v`. 4. Insérer dans `client` (5 lignes seulement) avec `INSERT INTO ... SELECT DISTINCT ... FROM v LIMIT 5`. 5. Vérifier : `SELECT * FROM client` → 5 lignes. 6.

Fermer (`con.close()`) et rouvrir. Vérifier que la base est vide (`:memory:` n'a pas de persistance — c'est l'erreur pédagogique attendue).

Barème (/15). 2 pt par étape + 5 pt bonus si l'étape 6 est comprise (`:memory:` ≠ fichier).

13. Exercices autonomes

- **Exercice 13.1 (15 min).** Listez les 3 modes d'ouverture de DuckDB (`:memory:`, fichier, `read_only`). Donnez un cas d'usage pour chacun.
- **Exercice 13.2 (30 min).** Reproduisez la chaîne DuckDB du § 8.2 sur le `quincaillerie_export.csv`. Vérifiez Q1, Q2, Q3. Notez les temps.
- **Exercice 13.3 (45 min).** Ouvrez la base SQLite `quincaillerie.sqlite` depuis Python, exécutez les 3 requêtes `controles_integrite.sql`. Notez 3 différences syntaxiques avec DuckDB.
- **Exercice 13.4 (60 min, optionnel).** Générez un `EXPORT DATABASE` de votre base DuckDB, ouvrez les fichiers `.parquet` produits avec `pandas.read_parquet()`. Commentez l'opération : est-ce portable ? est-ce lisible sans DuckDB ?

14. Correction détaillée

- **Exercice 13.1.** Trois modes : (a) `:memory:` — tests, exploration, pas de persistance ; (b) fichier `.duckdb` — production, persistance, sauvegarde ; (c) `read_only=True` — consultation seule (client graphique, audit, sécurité).
- **Exercice 13.2.** Temps attendus : import CSV ≈ 80 ms (24 000 lignes) ; insertion client ≈ 30 ms ; Q1 ≈ 5 ms. **Total ≈ 120 ms.** Sur l'atelier, c'est sub-second ; sur un poste de production, c'est aussi sub-second.
- **Exercice 13.3.** Différences attendues : SQLite utilise `INTEGER` (DuckDB aussi, mais peut utiliser `BIGINT`) ; SQLite n'a pas de `SHOW TABLES` (utiliser `sqlite_master`) ; SQLite n'a pas `EXPORT DATABASE` (utiliser `.backup`).
- **Exercice 13.4.** Opération portable : oui, les Parquet sont universels (`pandas`, `pyarrow`, `polars`, `Spark`, `Athena`, `BigQuery`). Opération lisible sans DuckDB : oui, Parquet est auto-descriptif. La seule chose perdue est la **vue** `ligne_vente_ttc` — il faut la redéfinir dans le `schema.sql` lors de la restauration.

15. Mini-projet M06.P3 — « L'import contrôlé du brut » (1 h)

Énoncé. En reprenant le script § 8.2, **compléter** pour atteindre le verdict du projet M06.P : 1. Ouvrir la base en mode fichier (`quincaillerie.duckdb`). 2. Créer les 7 tables du `schema_3fn.sql`. 3. Importer 24 000 ventes dans `vente` (filtrées). 4. Importer 5 000 lignes `ligne_vente` (à partir des mêmes données, 1 produit par vente). 5. **Vérifier** par les 3 requêtes `controles_integrite.sql`. 6. **Sauvegarder** par `EXPORT DATABASE 'backup_dir'`. 7. **Documenter** dans un rapport (1 page) : lignes par table, totaux, anomalies trouvées.

Critères de réussite.

1. La base `quincaillerie.duckdb` fait **< 2 Mo**.
2. Les **3 requêtes** renvoient 0/0/5.
3. Le **rapport** liste 4 anomalies trouvées (parmi les 142 montants texte, 8 doublons, 12 dates, 3 clients orphelins, ...) avec leur traitement.

4. La **sauvegarde** crée un dossier `backup_dir/` avec ≥ 5 fichiers Parquet.

Barème (/15). 3 pt par critère vérifié, 3 pt pour le rapport, 2 pt bonus pour la vérification cumulative (les 3 requêtes d'un coup).

16. Boîte à outils du chapitre

Boîte à outils. Sept objets, un seul but — vivre avec un SGBD sans drame :

1. **La chaîne DuckDB en 6 étapes** (§ 9) — installer, ouvrir, importer, insérer, vérifier, sauvegarder.
2. **Les 3 modes d'ouverture DuckDB** (§ 5.2) — `:memory:`, fichier, `read_only`.
3. **Les 2 voies d'insertion** (§ 5.4) — `INSERT INTO ... VALUES` (test) vs `INSERT INTO ... SELECT ... FROM read_csv_auto()` (import).
4. **Le PRAGMA foreign_keys = ON SQLite** (§ 6.1) — l'activation sans laquelle FK est fantôme.
5. **Les 5 différences DuckDB / SQLite** (§ 6) — natif, pragma, parquet, sauvegarde, CSV.
6. **Les 3 verbes de sauvegarde** — `EXPORT DATABASE` (DuckDB), `.backup` (SQLite), `pg_dump` (PostgreSQL).
7. **Le client graphique cité** (§ 5.5) — DBeaver, DataGrip, pgAdmin : utile pour 100 000 lignes, pas pour les tests.

17. Health Check (auto-correction, 6 questions, 15 min)

Q1. Quel est l'intérêt de `pip install duckdb --no-cache-dir` ? (Réponse : éviter que le cache `pip` ne grossisse de 100 Mo et sature le quota persistant de l'atelier.)

Q2. Pourquoi `all_varchar=true` à l'import CSV ? (Réponse : empêcher DuckDB autoloading de convertir silencieusement les colonnes sales — on veut garder la trace des défauts.)

Q3. Quel est le piège classique de SQLite ? (Réponse : `PRAGMA foreign_keys = ON` n'est pas activé par défaut, donc les FK sont des annotations non portées.)

Q4. Quand choisit-on DuckDB, SQLite, PostgreSQL ? (Réponse : DuckDB = analyse solo, SQLite = embarqué, PostgreSQL = multi-utilisateurs.)

Q5. Quelle commande DuckDB exporte toute la base ? (Réponse : `EXPORT DATABASE 'backup_dir'.`)

Q6. Pourquoi `:memory:` n'est-il pas utilisable en production ? (Réponse : la base est en mémoire RAM et perdue à la fermeture de la connexion — pas de persistance.)

Q7. Que font les 3 requêtes `controles_integrite.sql` ? (Réponse : Q1 vérifie l'intégrité référentielle (0 ligne) ; Q2 vérifie la cohérence des montants (0 ligne) ; Q3 calcule le total par catégorie (5 lignes).)

Q8. Quels sont les 3 verbes de sauvegarde ? (Réponse : `EXPORT DATABASE` pour DuckDB, `.backup` pour SQLite, `pg_dump` pour PostgreSQL.)

Q9. Citez 2 clients graphiques pour PostgreSQL. (Réponse : pgAdmin (officiel), DBeaver (universel), DataGrip (JetBrains).)

Q10. Pourquoi la commande `pg_dump` n'est-elle pas exécutable dans l'atelier ? (Réponse : parce que PostgreSQL n'est pas installé dans cet atelier — c'est la règle 7 du plan M05 reconduite pour M06 ; on cite sans exécuter.)

18. Résumé du chapitre

- **DuckDB** est un SGBD intégré (in-process), mono-utilisateur, gratuit. Il lit CSV, Parquet, JSON, fait du SQL ANSI.
- **SQLite** est un SGBD embarqué, natif Python 3.13, mono-utilisateur. Le `fichier.sqlite` est la base.
- **PostgreSQL** est un SGBD serveur, multi-utilisateurs, open source. C'est la référence de la production, **non installée** dans cet atelier (règle 7 du plan M05 reconduite).
- L'**import CSV** canonique est `INSERT INTO ... SELECT ... FROM read_csv_auto('fichier.csv', all_varchar=true)`.
- **Trois sauvegardes** : `EXPORT DATABASE` (DuckDB), `.backup` (SQLite), `pg_dump` (PostgreSQL).
- La **chaîne DuckDB en 6 étapes** (installer, ouvrir, importer, insérer, vérifier, sauvegarder) est l'épreuve de la pratique.

19. À retenir

À retenir. Le choix DuckDB / SQLite / PostgreSQL est une question de contraintes locales (utilisateurs, volumétrie, persistance), pas une question de goût. DuckDB en première intention pour l'analyste solo, SQLite pour embarquer, PostgreSQL pour partager. Le bon professionnel sait quand basculer : c'est la règle des 5 critères du C05 + la simplicité de mise en place (DuckDB : aucune ; SQLite : natif ; PostgreSQL : serveur à installer).

À retenir. `all_varchar=true` est le **garde-fou** de l'import CSV. Sans lui, DuckDB autoloade convertit silencieusement « 1 200 FCFA » en 1200, et la trace des défauts est **perdue**. Avec, la colonne reste `TEXT` jusqu'à la conversion contrôlée par script. C'est la même discipline que M04 §5.6 sur les formats : charger brut, transformer sous contrôle.

À retenir. Le client graphique (DBeaver, DataGrip, pgAdmin) est **cité non exécuté** dans cet atelier — c'est l'usage quotidien d'un analyste sur 100 000+ lignes, mais il n'est pas nécessaire pour un atelier de formation. Le CLI `duckdb fichier.duckdb` suffit pour les tests ; le client graphique est l'outil du confort, pas l'outil de la preuve.

20. Évaluation formative (auto-correction, 8 min)

Question 1. Citez les 3 SGBD du module et leur cas d'usage. (Réponse : DuckDB = analyste solo intégré ; SQLite = embarqué mono-utilisateur ; PostgreSQL = serveur multi-utilisateurs.)

Question 2. Quelle est la différence entre `EXPORT DATABASE` et `pg_dump` ? (Réponse : les deux exportent la base, mais DuckDB exporte en Parquet + SQL tandis que PostgreSQL exporte en SQL texte uniquement.)

Question 3. Pourquoi `PRAGMA foreign_keys = ON` est-il indispensable en SQLite ? (Réponse : sans ça, les FK sont des annotations non portées — la base ne refuse pas les orphelins.)

Question 4. Quel est le piège de `read_csv_auto` sans `all_varchar=true` ? (Réponse : DuckDB autoloade convertit silencieusement « 1 200 FCFA » en 1200 et masque le défaut métier.)

Question 5. Que fait la commande `EXPORT DATABASE 'backup_dir'` ? (Réponse : exporte toute la base DuckDB en Parquet (tables) + `schema.sql` (DDL), portable vers n'importe quel outil Parquet.)

Question 6. Citez 3 clients graphiques de PostgreSQL. (Réponse : pgAdmin, DBeaver, DataGrip.)

Question 7. Pourquoi ne pas exécuter PostgreSQL dans cet atelier ? (Réponse : la règle 7 du plan M05 reconduite pour M06 ; l'installation d'un serveur PostgreSQL est ≈ 150 Mo et 5-10 min, hors budget de l'atelier ; on cite sans exécuter.)

Question 8. Que se passe-t-il si on ferme une connexion `:memory:` ? (Réponse : toute la base est perdue — c'est le comportement attendu d'une base volatile.)

Module M06.C05 — Les autres mondes : NoSQL (document, clé-valeur, graphe, colonnes), datalake, entrepôt, lac

Outils : aucun outil à installer. Vocabulaire d'entretien. Durée indicative : 5 h. Niveau : N2. Prérequis : M06.C01, M06.C02, M06.C03.

L'idée du chapitre. Les quatre chapitres précédents ont installé le **SGBD relationnel** (DuckDB, SQLite, PostgreSQL). Le C05 regarde au-delà : ce qui vient autour, à côté et au-dessus du relationnel. Le C05 n'est pas un détour : c'est l'**ouverture professionnelle** du module. Un analyste qui ne connaît que les SGBD relationnels a une vision tronquée du paysage. Le module M06 lui donne les **5 mots d'entretien** (NoSQL, datalake, entrepôt, lac, lakehouse) et les **5 cas** où un autre modèle est meilleur. Le verdict du C05 est « l'analyste sait où il est, et où il n'est pas ».

Base de travail — pas de fichier livré pour ce chapitre. Le C05 est **purement descriptif**. Il n'enseigne aucun outil à installer (NoSQL n'est pas dans l'atelier), aucun pipeline à exécuter, aucune table à vérifier. C'est le chapitre qui prépare l'entretien d'embauche et le vocabulaire transversal du data engineer. Le mini-projet M06.P4 est le seul livrable « concret » : un glossaire de 200 mots qui couvre les 5 mots d'entretien.

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **définir** NoSQL, et distinguer ses **4 familles** (document, clé-valeur, graphe, colonnes) ;
- **expliquer** pourquoi NoSQL ne remplace pas le relationnel mais le complète ;
- **situer** datamart, datawarehouse (entrepôt), datalake (lac) et lakehouse dans le paysage ;
- **expliquer** les **5 mots d'entretien** : sharding, réplication, CAP, ETL vs ELT, ACID ;
- **reconnaître** le bon choix pour un cas d'usage (transactionnel vs analytique, structuré vs non-structuré) ;
- **rédiger** un glossaire de 200 mots sur les 5 mots d'entretien (mini-projet M06.P4).

2. Pourquoi cette notion est importante

- Le module M06 a installé le **relationnel** sur **50 % du paysage** (cf. C01 § 5.6). L'autre moitié est **NoSQL et Big Data**. Le C05 ferme la porte en montrant ce qui existe à côté — sans l'enseigner en profondeur, parce que c'est l'objet de modules ultérieurs (M11 « SQL avancé », M13 « Modélisation », M14 « Power BI »).
- **NoSQL** est un raccourci pour « Not only SQL ». Ce n'est pas « No SQL » (pas de SQL) ; c'est « Pas seulement SQL ». Le C05 enseigne les **4 familles** (document, clé-valeur, graphe, colonnes) et les **cas d'usage** de chacune.
- Le **datalake** et le **datawarehouse** (entrepôt) sont des concepts complémentaires, pas opposés. Le datawarehouse stocke des **données structurées** (et validées) ; le datalake stocke **tout** (CSV, JSON, images, logs). Le **lakehouse** combine les deux. C'est l'architecture moderne.

- Les **5 mots d'entretien** sont ce qui distingue un junior (qui les a lus) d'un senior (qui les a vécus). Le C05 les explique en **moins de 200 mots** par mot, pour que l'analyste junior puisse répondre en entretien sans bluff.
- Le **paysage** est ce qui permet de situer un projet : « ce projet est transactionnel, mono-utilisateur, structuré → SQLite » ; « ce projet est transactionnel, multi-utilisateurs, gouvernance forte → PostgreSQL » ; « ce projet est non-structuré, multi-utilisateurs, documente → MongoDB ». C'est la **compétence d'orientation** du professionnel.

3. Explication simple — la carte du monde des données

Le **paysage** se lit en **4 axes** :

1. **Structure** — les données sont-elles **typées** (SGBDR), **semi-structurées** (JSON, MongoDB), ou **non-structurées** (texte brut, images) ?
2. **Fréquence** — la base est transactionnelle (caisse, bancaire) ou analytique (rapports, agrégats) ?
3. **Utilisateurs** — 1 (analyste solo), quelques-uns (équipe), milliers (grand public) ?
4. **Gouvernance** — locale (un fichier), centralisée (schéma SQL), fédérée (multi-sites) ?

Le **résumé en une phrase** : « Un bon choix d'outil dépend de où on est sur ces 4 axes. »

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
NoSQL — Not only SQL	la famille des bases non-relationnelles : document, clé-valeur, graphe, colonnes larges.	confondre avec « pas de SQL » : NoSQL complète le relationnel, ne le remplace pas.
base document — document store	la base qui stocke des JSON-like (MongoDB, CouchDB). Chaque document est typé mais sans schéma imposé.	confondre avec « base non structurée » : MongoDB a des schémas implicites (validés au niveau applicatif).
base clé-valeur — key-value store	la base qui stocke des paires clé → valeur (Redis, DynamoDB). Ultra-rapide en lecture/écriture simple.	confondre avec « pas de clé » : la clé est l'identité, la valeur est opaque.
base graphe — graph database	la base qui stocke des nœuds + arêtes (Neo4j, JanusGraph). Optimisée pour les parcours de graphe.	confondre avec « table de jointure » : un graphe est une structure dédiée, pas une astuce relationnelle.
base colonnes — wide-column store	la base qui stocke des colonnes par famille (Cassandra, Bigtable). Optimisée pour les lectures massives en colonne.	confondre avec « colonne SQL » : c'est une organisation physique différente, pas un type logique.
MongoDB — MongoDB	le SGBD document le plus populaire, JSON natif, schéma flexible, écrit en C++. Utilisé par 50 % des startups en 2024.	croire qu'il remplace PostgreSQL : MongoDB a des transactions depuis 4.0 (2018), mais reste plus lent sur les jointures.
Redis — Redis	le SGBD clé-valeur ultra-rapide, mémoire RAM, persistance sur disque en option. Utilisé pour le cache, les files, les compteurs.	confondre avec « base principale » : Redis est un cache, pas une base canonique.
Neo4j — Neo4j	le SGBD graphe le plus populaire, écrit en Java, requête en Cypher (<code>MATCH (a)-[:KNOWS]->(b) RETURN b.name</code>).	croire qu'il est réservé aux réseaux sociaux : Neo4j sert aussi pour la traçabilité alimentaire, les graphes de dépendance, etc.
Cassandra — Apache Cassandra	le SGBD colonnes larges distribué, écrit en Java, optimisé pour l'écriture massive. Utilisé par Netflix, Apple, Uber.	confondre avec « colonne » : Cassandra organise les données par famille de colonnes, pas par colonnes SQL.

Français — English	Définition simple	Piège à éviter
NewSQL — NewSQL	la famille des SGBD relationnels distribués (CockroachDB, Spanner, YugabyteDB). Le meilleur des deux mondes : SQL + scalable horizontalement.	confondre avec « PostgreSQL distribué » : NewSQL ajoute la distribution (sharding automatique) sans abandonner le SQL.
datawarehouse — data warehouse, DWH	l' entrepôt de données : un SGBD analytique (SQL Server, Snowflake, BigQuery, Redshift) où les rapports sont calculés.	confondre avec « base de production » : le DWH est séparé de la base transactionnelle.
datalake — data lake, lac	le lac de données : un stockage brut (S3, HDFS) qui contient des CSV, JSON, Parquet, images, logs, sans validation préalable.	confondre avec « SGBD » : un lac ne valide pas, c'est juste un gros disque.
lakehouse — lakehouse	l' architecture hybride : un lac + un entrepôt, où les données sont stockées brutes et validées en SQL via Delta Lake ou Apache Iceberg.	croire qu'il faut choisir : lakehouse est les deux à la fois.
ETL — Extract, Transform, Load	le pipeline classique : on transforme les données avant de les charger (par exemple, dans un SGBDR).	confondre avec « copie de fichiers » : ETL a une logique de nettoyage.
ELT — Extract, Load, Transform	le pipeline moderne : on charge d'abord (dans le lac), puis on transforme dans le SGBD analytique.	croire qu'il faut choisir : ELT est le standard moderne (datalake → entrepôt).
sharding — sharding, partitionnement horizontal	la répartition des données sur plusieurs machines (ex. : les clients A-M sur le serveur 1, N-Z sur le serveur 2). C'est la scalabilité horizontale des SGBD distribués.	confondre avec « partition verticale » : sharding = par lignes, partition verticale = par colonnes.
réplication — replication	la copie asynchrone (ou synchrone) des données sur plusieurs machines. Maître-esclave (un maître, plusieurs copies) ou multi-maître (plusieurs maîtres qui se synchronisent).	confondre avec « sauvegarde » : la réplication est temps réel, la sauvegarde est périodique.
CAP — CAP theorem, Brewer's theorem	l'impossibilité d'avoir simultanément Cohérence, Disponibilité, Tolérance au partitionnement dans un SGBD distribué. Il faut en choisir 2 sur 3.	croire qu'on peut tout avoir : c'est mathématiquement impossible, le théorème de Brewer est prouvé.
ACID — Atomicity, Consistency, Isolation, Durability	les 4 propriétés d'une transaction SGBD (cf. C01 §4 : atomique, cohérent, isolé, durable). Tous les SGBDR respectent ACID.	confondre avec BASE (cf. ci-dessous) : NoSQL « sacrifie » souvent ACID pour la disponibilité (BASE).
BASE — Basically Available, Soft state, Eventually consistent	l' alternative à ACID pour les SGBD distribués NoSQL : on accepte une cohérence différée au profit de la disponibilité.	confondre avec « pas de cohérence » : BASE est cohérent à terme (les lectures lointaines convergent), pas « n'importe quoi ».

Définition. Le **NoSQL** (Not only SQL) est la famille des bases **non-relationnelles**, optimisées pour des cas où le relationnel est maladroit : documents semi-structurés (MongoDB), paires clé-valeur ultra-rapides (Redis), graphes de relations (Neo4j), colonnes larges distribuées (Cassandra). NoSQL ne remplace pas le relationnel ; il le complète. La règle du module est « choisir NoSQL quand le relationnel est inadapté, pas par mode. » — English : Not only SQL.

Définition. Le **datawarehouse** (entrepôt) est un SGBD **analytique** où les données validées sont stockées pour les rapports (tableaux de bord, SQL analytique, OLAP). Le **datalake** (lac) est un stockage brut où les données sont conservées telles quelles (CSV, JSON, images, logs). Le **lakehouse** combine les deux : les données sont stockées brutes et validées en SQL via des couches comme Delta Lake ou Apache Iceberg. La règle du module est DWH pour les rapports, lac pour l'exploration, lakehouse pour les deux. — English : data warehouse, data lake, lakehouse.

Définition. Le **théorème CAP** (Brewer, 2000) énonce qu'un SGBD **distribué** (sur plusieurs machines) ne peut pas avoir simultanément **Cohérence**, **Disponibilité**, **Tolérance au partitionnement**. Le choix se fait entre **CP** (cohérence + partitionnement, e.g. HBase) ou **AP** (disponibilité + partitionnement, e.g. Cassandra) ou **CA** (cohérence + disponibilité, impossible en distribué — c'est l'apanage du SGBD centralisé). Le module le cite pour que l'analyste sache pourquoi NoSQL est différemment consistant. — English : CAP theorem.

Attention. **NoSQL ne supprime pas le schéma** : il le rend implicite. Sur MongoDB, le document `{nom: "Alice", age: 30}` est valide parce que l'application le crée ainsi. Si on change le code applicatif, les anciens documents restent dans l'ancien format. La **discipline** est de documenter le schéma attendu dans l'application, pas dans la base. C'est l'inverse de la discipline relationnelle, où la base porte le schéma (`CREATE TABLE`). Pour un junior, c'est une surprise ; pour un senior, c'est une responsabilité.

Attention. Le **sharding** est une décision **irréversible**. Si on commence avec un sharding par `id_client`, on ne peut pas revenir en arrière sans **réindexer toute la base**. Le piège classique est de sharder trop tôt (avant d'avoir 100 millions de lignes, le SGBD centralisé tient). La règle du module est sharder quand le SGBD centralisé ne tient plus — pas avant. Sans cette discipline, on prend une décision de 6 mois qu'on regrettera.

5. Cours approfondi — les 4 familles NoSQL

5.1 Document store (MongoDB, CouchDB)

Cas d'usage. Données **semi-structurées** (JSON, BSON, hiérarchiques) où le **schéma change souvent**. Exemples : profils utilisateurs (chacun avec des champs différents), catalogues produits (attributs variables), IoT (capteurs divers).

Exemple MongoDB.

```
db.users.insertOne({
  nom: "Alice",
  age: 30,
  adresses: [
    {ville: "Koudougou", pays: "BF"},
    {ville: "Bobo", pays: "BF"}
  ],
  preferences: {langues: ["fr", "moore"], fidelite: "OR"}
});
```

Le **schéma** est implicite dans la structure du document ; c'est le code applicatif qui le valide.

5.2 Clé-valeur (Redis, DynamoDB)

Cas d'usage. Cache, sessions, files d'attente, compteurs. La rapidité en lecture/écriture simple prime. Exemples : panier d'achat (`id_session` → liste de produits), cache HTTP (URL → réponse), leaderboard (`id_joueur` → score).

Exemple Redis.

```
SET user:1000 "Alice"      # clé → valeur
GET user:1000             # "Alice"
INCR compteur:visites:home # +1 atomique
HSET user:1000 nom "Alice" age 30 # hash (sous-clés)
```

Redis vit en mémoire RAM : c'est la base la plus rapide, et c'est aussi la plus volatile (les données sont perdues au redémarrage, sauf configuration `appendonly`).

5.3 Graphe (Neo4j, JanusGraph)

Cas d'usage. Réseaux sociaux, dépendances logicielles, traçabilité alimentaire, graphes de connaissances, lutte anti-fraude. La **structure de graphe** (nœuds + arêtes) est native : une requête qui parcourt 3 niveaux est directe (pas de 3 jointures).

Exemple Neo4j (Cypher).

```
MATCH (a:Person)-[:KNOWS]->(b)-[:KNOWS]->(c)
WHERE a.name = 'Alice'
RETURN c.name, count(*) AS amis_de_amis
```

Le **Cypher** est déclaratif : on décrit le **parcours**, on ne code pas l'algorithme. C'est 10 × plus court qu'une jointure SQL équivalente.

5.4 Colonnes larges (Cassandra, Bigtable, HBase)

Cas d'usage. Écriture massive en continu (capteurs IoT, logs de serveurs, historique de prix). Le **modèle** est colonnes par famille : on regroupe les colonnes par « famille » (par exemple, `info_personnelles`, `metriques`, `preferences`) et on charge par famille.

Exemple Cassandra (CQL).

```
CREATE TABLE users (
  id_user    UUID PRIMARY KEY,
  nom        TEXT,
  age        INT,
  ville      TEXT
) WITH COMPACT STORAGE;
```

Cassandra est optimisée pour **l'écriture** ; la lecture par clé est rapide, mais le scan par colonne est moins efficace que sur un SGBD analytique.

6. Cours approfondi — datamart, datawarehouse, datalake, lakehouse

6.1 Le datawarehouse (l'entrepôt, ~30 ans d'histoire)

- **Origine** : années 1990 (Bill Inmon, Ralph Kimball).
- **Concept** : un SGBD analytique où les données sont **validées, agrégées, et orientées sujet**.
- **Outils** : Teradata, Oracle Exadata, SQL Server, Snowflake, BigQuery, Redshift.

- **Cas d'usage** : rapports consolidés (CA par mois par rayon), OLAP, indicateurs de performance — KPI (Key Performance Indicator).

6.2 Le datamart (le sous-entrepôt)

- **Concept** : un sous-ensemble du DWH, orienté **un sujet métier** (finance, marketing, RH).
- **Avantage** : plus simple, plus rapide à mettre en place.
- **Inconvénient** : risque de silos (chaque datamart a sa propre logique).

6.3 Le datalake (le lac, ~10 ans d'histoire)

- **Origine** : 2010 (James Dixon, Pentaho).
- **Concept** : un stockage **brut** (S3, HDFS, blob Azure) où les CSV, JSON, Parquet, images, logs sont conservés telles quelles, sans validation préalable.
- **Avantage** : on peut tout stocker (logs de 5 ans, archives, médias).
- **Inconvénient** : c'est un marécage (swamp) si on ne le gère pas. La phrase consacrée est « datalake » → « data swamp » en 6 mois.

6.4 Le lakehouse (la combinaison ~5 ans)

- **Origine** : 2020 (Databricks Delta Lake, Apache Iceberg, Apache Hudi).
- **Concept** : un lac avec une **couche de validation SQL**. Les données sont stockées brutes, mais les requêtes SQL passent par une couche qui impose un schéma (sans modifier le fichier d'origine).
- **Outils** : Databricks, Snowflake, BigQuery, DuckDB (sur Parquet natif).
- **Avantage** : le meilleur des deux mondes.

7. Cours approfondi — les 5 mots d'entretien

7.1 Sharding (partitionnement horizontal)

Le **sharding** répartit les lignes d'une table sur **plusieurs machines**. Chaque machine (nœud) porte un **sous-ensemble** des lignes. Exemple : 1 milliard de clients, répartis en 10 nœuds de 100 millions chacun. Le SGBD route la requête vers le bon nœud en fonction de la **clé de sharding** (souvent `id_user % 10`).

Prérequis : 100 millions de lignes minimum (en dessous, le SGBD centralisé tient).

7.2 Réplication (copie asynchrone)

La **réplication** copie les données d'une machine **maître** vers une ou plusieurs machines **esclaves**. C'est l'inverse du sharding : on copie toute la base, pas une partie. C'est le mécanisme de la **haute disponibilité** : si le maître tombe, un esclave prend le relais.

Modes : synchrone (la transaction attend que l'esclave confirme) ou asynchrone (la transaction commit sur le maître, l'esclave copie en arrière-plan, avec un délai).

7.3 Théorème CAP (Brewer)

Un SGBD **distribué** ne peut pas avoir simultanément : - **Cohérence** (tous les nœuds voient la même valeur au même moment) ; - **Disponibilité** (chaque requête reçoit une réponse, succès ou échec) ; - **Tolérance au partitionnement** (le système continue à fonctionner même si un nœud est isolé).

Le choix est forcé : un SGBD distribué fait 2 sur 3. **CP** (cohérence + partition, e.g. HBase) ou **AP** (disponibilité + partition, e.g. Cassandra) ou **CA** (impossible en distribué — c'est le SGBD centralisé classique).

7.4 ACID vs BASE (modèle transactionnel)

Définition. Le **modèle ACID** (Atomicity, Consistency, Isolation, Durability — atomique, cohérent, isolé, durable) est le contrat des transactions SGBDR : chaque transaction est garantie atomique (tout ou rien), cohérente (la base reste valide), isolée (les autres transactions ne voient pas l'état intermédiaire), durable (la modification survit à un crash). Le modèle **BASE** (Basically Available, Soft state, Eventually consistent) est le modèle alternatif pour les SGBD distribués NoSQL : on accepte une cohérence différée au profit de la disponibilité. ACID pour le bancaire, BASE pour l'IoT ou le cache. C'est le choix du C05.

ACID : Atomicité, Cohérence, Isolation, Durabilité. C'est le modèle **relationnel** : chaque transaction est garantie isolée des autres et durable.

BASE : Basically Available, Soft state, Eventually consistent. C'est le modèle **NoSQL distribué** : on accepte une cohérence différée au profit de la disponibilité.

Propriété	ACID	BASE
Cohérence	immédiate	à terme
Disponibilité	réduite (verrous)	élevée
Cas d'usage	bancaire, transactionnel	IoT, cache, analytique

7.5 ETL vs ELT

ETL (Extract, Transform, Load) — pipeline classique. On extrait, on transforme (dans un outil dédié comme Informatica), puis on charge dans l'entrepôt. La transformation est **hors** du SGBD analytique.

ELT (Extract, Load, Transform) — pipeline moderne. On extrait, on charge brut (dans le lac), puis on transforme dans le SGBD analytique (BigQuery, Snowflake). Le SGBD analytique **est** la machine de transformation.

8. Exemple concret — le verdict du paysage

Pour le projet M06.P « La base de la quincaillerie », où se trouve la base ?

Axe	Valeur	Outil adapté
Structure	typée (3FN, 7 tables)	SGBDR (DuckDB / SQLite / PostgreSQL)
Fréquence	analytique (rapports hebdo)	SGBDR analytique (DWH) si volumétrie
Utilisateurs	1 analyste (Koudougou)	DuckDB intégré
Gouvernance	locale (un fichier)	DuckDB + EXPORT DATABASE quotidien

Le verdict : DuckDB est le bon outil. Le projet M06.P est 100 % relationnel. NoSQL et Big Data sont hors de ce projet, mais restent dans **l'horizon professionnel** de l'analyste qui veut grandir.

9. Démonstration pas à pas — les 5 mots d'entretien

9.1 Préparer une réponse à « C'est quoi le sharding ? »

« Le sharding est la répartition horizontale des lignes d'une table sur plusieurs machines. C'est la scalabilité des SGBD qui grossissent au-delà du téraoctet. Le piège est de sharder trop tôt en dessous de 100 millions de lignes, le SGBD centralisé tient, et sharding est irréversible. » (58 mots)

9.2 Préparer une réponse à « ACID ou BASE ? »

« ACID pour les transactions bancaires (chaque virement doit être garanti). BASE pour les capteurs IoT (on préfère perdre 1 % des lectures plutôt que bloquer le capteur). Le théorème CAP nous dit qu'on ne peut pas tout avoir en distribué. » (49 mots)

9.3 Préparer une réponse à « DWH ou datalake ? »

« Les deux, en lakehouse. Le datalake stocke le brut ; le DWH stocke le validé ; le lakehouse garde les deux couches et permet des requêtes SQL sur le brut via Parquet. C'est l'architecture moderne. » (39 mots)

Les 5 mots d'entretien se préparent en **moins de 60 mots chacun**. C'est la règle du « dire peu, dire juste » (cf. M03.C01 § 8 sur la communication).

10. Erreurs fréquentes

- **Croire que NoSQL remplace SQL** : c'est l'erreur de compréhension n°1. NoSQL complète le relationnel, ne le remplace pas. La règle est SQL pour 80 % des cas, NoSQL pour 20 %.
- **Sharder trop tôt** : la décision est irréversible. La règle est sharder quand le SGBD centralisé ne tient plus (1 To+).
- **Confondre datalake et DWH** : le lac stocke du brut, l'entrepôt stocke du validé. Le lakehouse combine les deux.
- **Croire que MongoDB est plus rapide que PostgreSQL** : faux dans 80 % des cas. MongoDB est rapide pour les documents JSON ; sur les jointures classiques, PostgreSQL gagne.
- **Oublier ACID dans une transaction bancaire** : c'est l'erreur classique. La règle est toujours ACID pour les transactions critiques (banque, paie, facturation).
- **Confondre réplication et sauvegarde** : la réplication est temps réel, la sauvegarde est périodique. Les deux sont complémentaires (la réplique peut tomber en panne au même moment que le maître, la sauvegarde non).
- **Croire que « BASE = pas de cohérence »** : c'est faux. BASE = cohérent à terme (les lectures lointaines convergent), pas « n'importe quoi ». C'est une nuance qui compte en entretien.

À retenir. Les 7 erreurs ci-dessus sont les **7 pièges** qu'un analyste junior fait en entretien. Le test du C05 est pouvoir répondre aux 5 mots en moins de 60 mots chacun.

11. Bonnes pratiques professionnelles

Dans les faits. L'anecdote qui justifie la prudence sur le datalake : un cadre d'une grande banque française disait en 2018 « nous avons 14 pétaoctets de datalake, et nous ne savons pas ce qu'il y a dedans ». C'est ce qu'on appelle un **data swamp** : un lac sans gouvernance qui accumule les fichiers sans qu'on sache les utiliser. La parade est imposer un schéma implicite dès l'ingestion (par exemple Delta Lake), avec un petit fichier `_schema.json` qui déclare les colonnes attendues. Sans cette discipline, le lac devient un marécage en 6-12 mois. Le coût d'un lakehouse bien géré est équivalent à celui d'un DWH bien géré ; le coût d'un lac mal géré est infiniment supérieur (parce qu'on ne peut plus rien en extraire).

- **Choisir NoSQL quand le relationnel est inadapté**, pas par mode. Le réflexe junior est « on va essayer MongoDB » ; le réflexe senior est « quelle est la donnée ? quelle est la requête ? choisir ensuite. »
- **Documenter le paysage** dans le README du projet (DWH, lac, lakehouse) pour que les nouveaux venus s'orientent.
- **Connaître les 5 mots d'entretien** par cœur (sharding, réplication, CAP, ACID/BASE, ETL/ELT), en moins de 60 mots chacun.
- **Différencier OLTP et OLAP** : OLTP = transactionnel (caisse), OLAP = analytique (DWH).
- **Savoir sharder** (en théorie) sans sharder en pratique avant 100 millions de lignes.

Conseil professionnel. Un entretien data, c'est 70 % de SQL et 30 % de paysage. Le C05 installe les 30 % qui manquent à un junior qui ne connaît que SQL.

12. Exercice guidé — préparer 5 réponses d'entretien (30 min, /15)

Objectif. Mobiliser les 5 mots d'entretien dans **des phrases courtes et justes**.

Énoncé. Pour chacun des 5 mots (sharding, réplication, CAP, ACID vs BASE, ETL vs ELT), rédigez **une réponse de 50-80 mots** qui : - donne la définition ; - donne un cas d'usage ; - cite un outil qui l'illustre.

Barème (/15). 3 points par mot (définition 1, cas 1, outil 1).

13. Exercices autonomes

- **Exercice 13.1 (15 min).** Dessinez la **carte du paysage** (§ 6) en 4 colonnes (datamart, DWH, datalake, lakehouse) avec : structure, fréquence, gouvernance, coût.
- **Exercice 13.2 (30 min).** Pour chaque famille NoSQL (document, clé-valeur, graphe, colonnes), trouvez **un cas d'usage concret dans votre entreprise** (ou dans une startup typique de la formation). Justifiez le choix.
- **Exercice 13.3 (45 min).** Prenez une grande entreprise (Netflix, Uber, Airbnb) et trouvez où ils utilisent chaque famille NoSQL. Cette recherche peut se faire sur le blog tech de l'entreprise.
- **Exercice 13.4 (60 min, optionnel).** Lisez l'article Wikipedia sur le théorème CAP et rédigez un résumé de 100 mots qui distingue CP, AP, CA avec un outil par catégorie.

14. Correction détaillée

- **Exercice 13.1.** Datamart : un sujet, rapide à monter, silos possibles. DWH : tous les sujets, unifié, plus long à monter. Datalake : tout, brut, marécage en 6 mois sans gouvernance. Lakehouse : les deux, moderne, nécessite Delta Lake ou Iceberg.
- **Exercice 13.2.** Document (MongoDB) : profils utilisateurs, attributs variables. Clé-valeur (Redis) : cache de session, panier. Graphe (Neo4j) : réseau social, recommandation. Colonnes (Cassandra) : historique de capteurs IoT, logs massifs.
- **Exercice 13.3.** Netflix : Cassandra pour le streaming, EVCache (clé-valeur maison) pour le cache. Uber : Schemaless (MySQL modifié) + Cassandra + Redis. Airbnb : MySQL + Druid (OLAP). Le schéma se précise dans la presse tech.
- **Exercice 13.4.** CAP : (a) **CP** = cohérence + partitionnement, HBase ; (b) **AP** = disponibilité + partitionnement, Cassandra ; (c) **CA** = cohérence + disponibilité, impossible en distribué, c'est l'apanage du SGBD centralisé (PostgreSQL, MySQL classique).

15. Mini-projet M06.P4 — « Le glossaire des 5 mots d'entretien » (1 h)

Énoncé. Rédigez un **glossaire** de 200 mots (± 20) qui couvre les 5 mots d'entretien, en respectant les contraintes suivantes :

- 5 entrées (sharding, réplication, CAP, ACID/BASE, ETL/ELT).
- Chaque entrée fait **30-50 mots**.
- L'entrée est **structurée** : définition, cas d'usage, outil, piège.
- Le glossaire totalise **200 $\pm$ 20 mots**.
- Le glossaire est livré en `03_exercices/dossier_M06/glossaire_M06P4.md`.

Critères de réussite.

1. Les **5 entrées** sont présentes (1).
2. Chaque entrée a **définition + cas + outil + piège** (1 par champ, 4 points = 4).
3. Le total mots est **180-220** (1).
4. **Pas de phrase de plus de 50 mots** par entrée (1).
5. Le glossaire est **vérifiable** : un relecteur peut le lire sans bloquer.

Barème (/10). 2 pt par critère sauf le 4 et 5 (1 pt chacun).

16. Boîte à outils du chapitre

Boîte à outils. Sept objets, un seul but — s’orienter dans le paysage des données :

1. **Les 4 familles NoSQL** (§5) — document (MongoDB), clé-valeur (Redis), graphe (Neo4j), colonnes larges (Cassandra).
2. **Les 4 concepts de stockage** (§6) — datamart, DWH, datalake, lakehouse.
3. **Les 5 mots d’entretien** (§7) — sharding, réplication, CAP, ACID/BASE, ETL/ELT.
4. **La matrice du projet M06.P** (§8) — où se trouve la quincaillerie : structure / fréquence / utilisateurs / gouvernance = DuckDB.
5. **Le CAP theorem** (§7.3) — 2 sur 3, jamais 3 ; c’est mathématiquement impossible.
6. **Le tableau ACID vs BASE** (§7.4) — bancaire = ACID, IoT = BASE.
7. **Le mini-projet M06.P4** — le glossaire de 200 mots, l’épreuve du C05.

17. Résumé du chapitre

- Le **NoSQL** est Not only SQL — la famille des bases non-relationnelles (document, clé-valeur, graphe, colonnes larges). Il complète le relationnel, ne le remplace pas.
- **DWH** (entrepôt) stocke du validé ; **datalake** (lac) stocke du brut ; **lakehouse** combine les deux.
- Les **5 mots d’entretien** (sharding, réplication, CAP, ACID/BASE, ETL/ELT) sont la discipline du C05. Le professionnel qui les connaît passe l’entretien data.
- Le **projet M06.P** est 100 % relationnel (structure typée, 1 utilisateur, gouvernance locale, fréquence analytique). DuckDB est le bon choix. NoSQL et Big Data sont hors de ce projet mais dans l’horizon professionnel.
- Le **mini-projet M06.P4** produit un glossaire de 200 mots sur les 5 mots d’entretien.

18. À retenir

À retenir. NoSQL complète le relationnel, ne le remplace pas. La règle du module est « choisir NoSQL quand le relationnel est maladroit », pas par mode. Les 4 familles (document, clé-valeur, graphe, colonnes) couvrent 4 cas d’usage distincts, et c’est cette **diversité** qui fait la force du paysage.

À retenir. Les 5 mots d’entretien se préparent en **moins de 60 mots chacun**. C’est la règle du « dire peu, dire juste ». Un entretien data est 70 % SQL + 30 % paysage, et le C05 installe les 30 % qui manquent.

19. Évaluation formative (auto-correction, 8 min)

Question 1. Citez les 4 familles NoSQL. (Réponse : document, clé-valeur, graphe, colonnes larges.)

Question 2. Quelle est la différence entre un DWH et un datalake ? (Réponse : le DWH stocke du validé ; le datalake stocke du brut, sans validation préalable.)

Question 3. Que dit le théorème CAP ? (Réponse : un SGBD distribué ne peut pas avoir simultanément Cohérence, Disponibilité, Tolérance au partitionnement — il faut en choisir 2 sur 3.)

Question 4. Donnez un cas d'usage de MongoDB. (Réponse : profils utilisateurs avec attributs variables, catalogues produits, IoT.)

Question 5. Donnez un cas d'usage de Redis. (Réponse : cache de session, panier d'achat, compteurs temps réel.)

Question 6. Donnez un cas d'usage de Neo4j. (Réponse : réseau social, traçabilité alimentaire, graphe de dépendances logicielles.)

Question 7. Donnez un cas d'usage de Cassandra. (Réponse : historique de capteurs IoT, logs massifs, écriture en continu.)

Question 8. Quelle est la différence entre ETL et ELT ? (Réponse : ETL transforme hors du SGBD analytique ; ELT charge brut et transforme dans le SGBD analytique.)

Question 9. Quand sharder ? (Réponse : au-delà de 1 To de données ou de 100 millions de lignes ; en dessous, le SGBD centralisé tient. Sharding est irréversible.)

Question 10. Quelle est la différence entre ACID et BASE ? (Réponse : ACID est le modèle relationnel (transaction bancaire, isolation, durabilité) ; BASE est le modèle NoSQL distribué (disponibilité élevée, cohérence à terme).)